

▼ Remote procedure call

1	◆ Remote Procedure Call
2	◆ Remote Procedure Call (RPC)
▼ RPC basics	
3	◆ Filesystem + RPC: a form of distributed filesystem
4	◆ RPC vs. sockets API (another alternative for communication)
5	◆ RPC: remote procedure call
6	◆ Example of RPC
7	◆ Call GET_TIME in the single-machine case
8	◆ Single-machine call vs. distributed call (RPC)
9	◆ Handwritten code: client
10	◆ Handwritten code: server
11	◆ RPC simplifies the implementation of remote calls
12	◆ RPC: a complete calling process
13	◆ RPC stub
14	◆ Client program using RPC
15	◆ Server program using RPC
16	◆ Question: what is inside a message?
17	◆ RPC request message
18	◆ RPC reply message
19	◆ Why does a request need an xid?
▼ Passing data between client & server	
20	◆ Parameter passing
21	◆ Parameter passing is challenging across machines
22	◆ Parameter passing is more challenging
23	◆ Representing data: encoding
24	◆ Why not use language-specific formats?
25	◆ System requirements for encoding/decoding
26	◆ Standardized encoding: JSON, XML, etc.
27	◆ Binary formats
28	◆ Binary formats: schema
29	◆ The BinaryProtocol of Thrift
30	◆ Being more compact: Thrift CompactProtocol
31	◆ Schema simplifies supporting compatibility
32	◆ Short summary of parameter passing in RPC
33	◆ Automatic stub generation
34	◆ Transport protocol of RPC
35	◆ Short summary of encoding & decoding data for RPC
36	◆ Short summary: RPC so far
▼ When RPC meets failure	
37	◆ When RPC meets failures
38	◆ RPC != PC
39	◆ When RPC meets failures
40	◆ RPC semantics
41	◆ Birrell's RPC semantics: how
42	◆ Birrell's RPC semantics: why useful
43	◆ RPC semantics
44	◆ Cases of idempotence
45	◆ Idempotence in RPC
46	◆ RPC semantics
47	◆ Ideal RPC semantics: exactly-once
48	◆ Idempotence in RPC
49	◆ Ideal RPC semantics: exactly-once
▼ Put it all together	
50	◆ Put it all together: RPC system components
▼ RPC implementations	
51	◆ Case study: Sun RPC / reply cache
52	◆ Case study: gRPC / Protobuf
53	◆ Case study: gRPC / retry
54	◆ References

Remote Procedure Call

IPADS, Shanghai Jiao Tong University

Remote Procedure Call (RPC)

Implementing Remote Procedure Calls

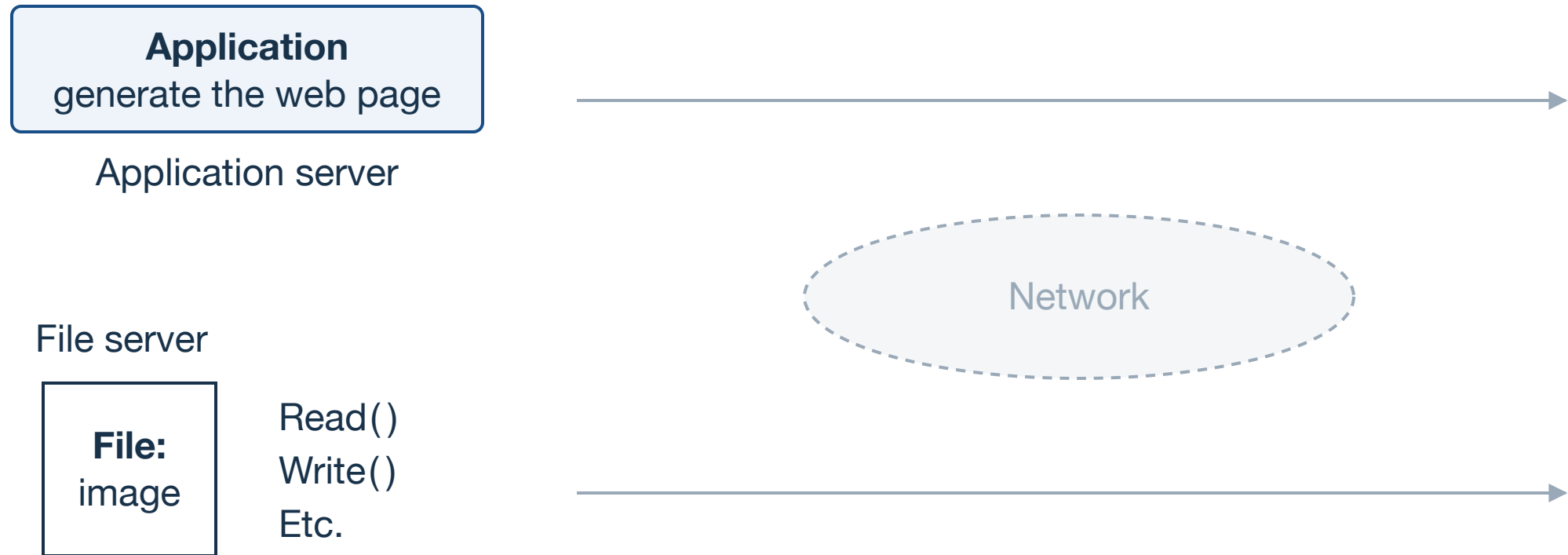
ANDREW D. BIRRELL and BRUCE JAY NELSON
Xerox Palo Alto Research Center

Remote procedure calls (RPC) appear to be a useful paradigm for providing communication across a network between programs written in a high-level language. This paper describes a package providing a remote procedure call facility, the options that face the designer of such a package, and the decisions we made. We describe the overall structure of our RPC mechanism, our facilities for binding RPC clients, the transport level communication protocol, and some performance measurements. We include descriptions of some optimizations used to achieve high performance and to minimize the load on server machines that have many clients.

 **Birrell & Nelson, 1984:** Remote procedure calls (RPC) appear to be a **useful paradigm** for providing communication across a network.

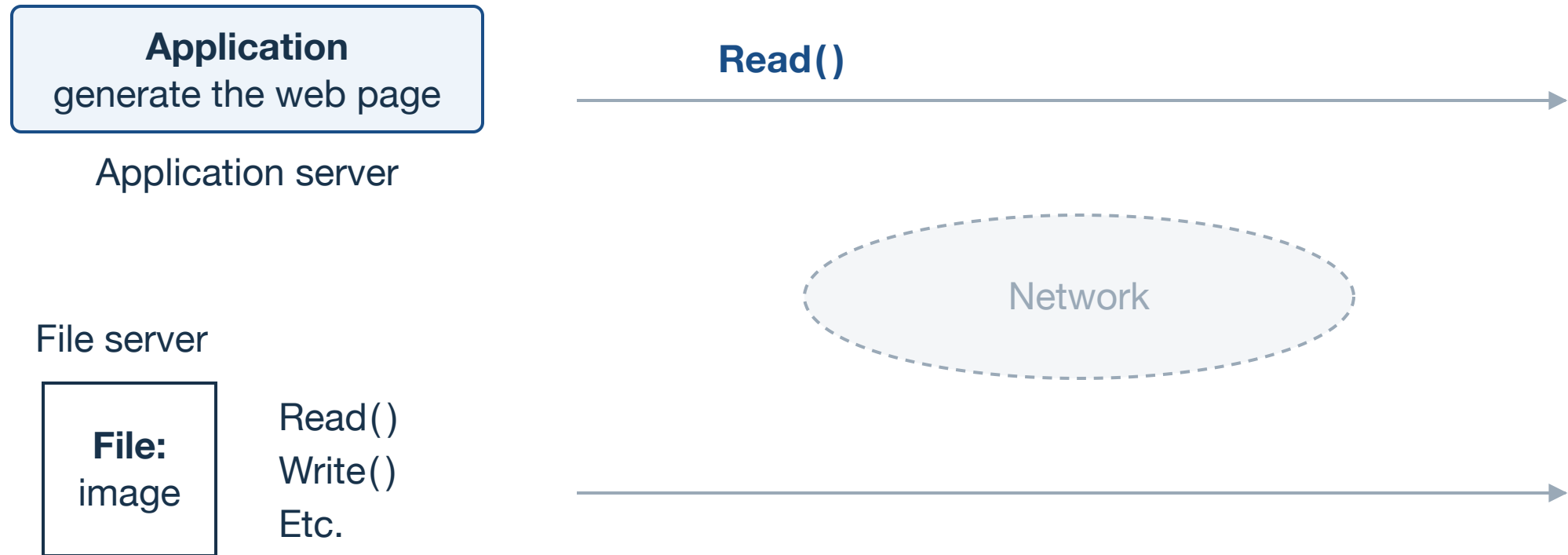
Filesystem + RPC: a form of distributed filesystem

Calling a function on a remote server like a local one (without any modification)



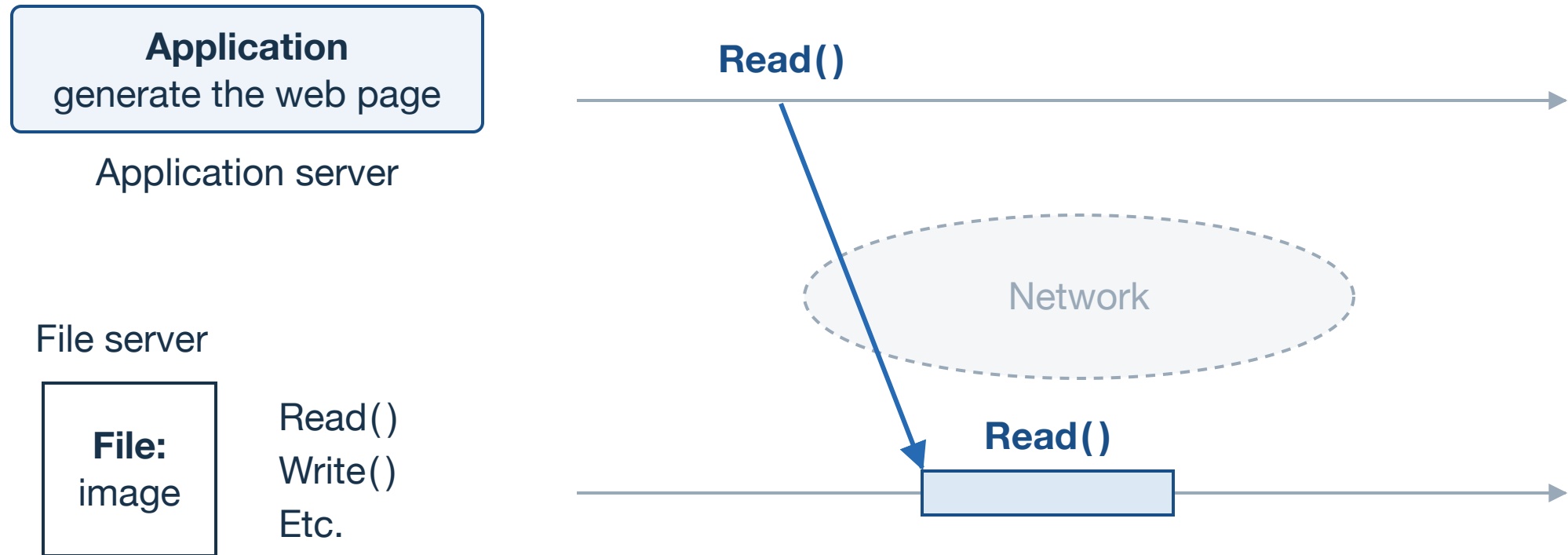
Filesystem + RPC: a form of distributed filesystem

Calling a function on a remote server like a local one (without any modification)



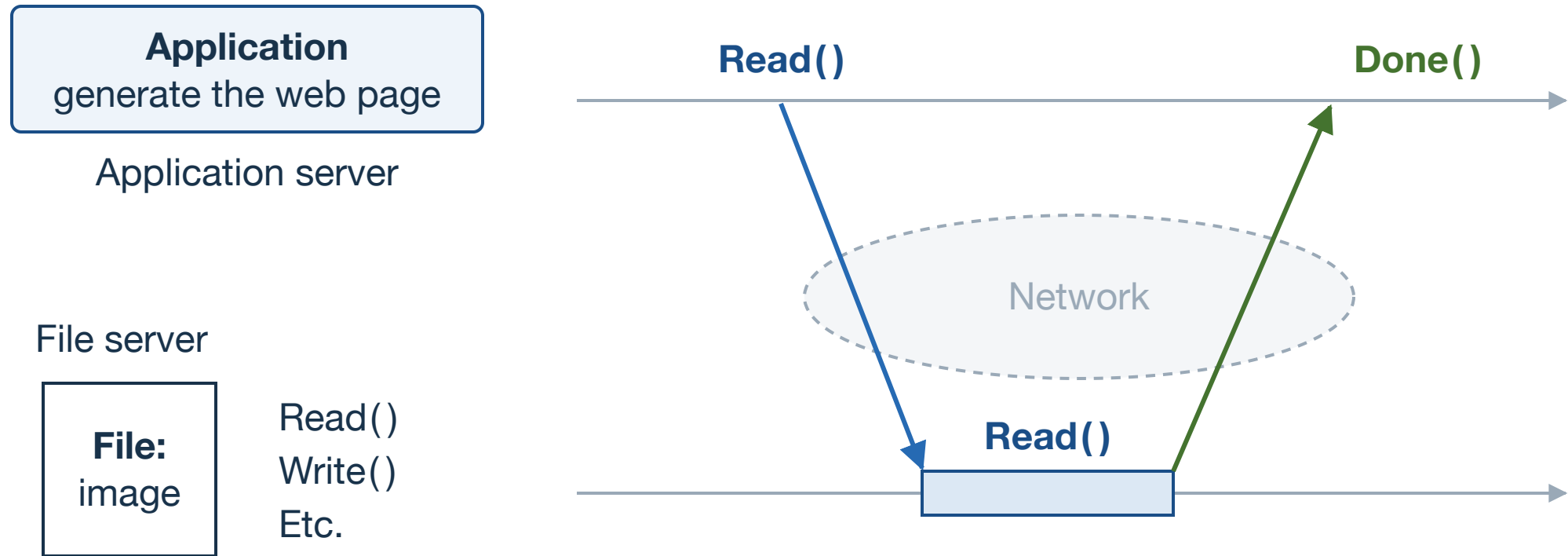
Filesystem + RPC: a form of distributed filesystem

Calling a function on a remote server like a local one (without any modification)



Filesystem + RPC: a form of distributed filesystem

Calling a function on a remote server like a local one (without any modification)



RPC vs. sockets API (another alternative for communication)

The sockets interface forces a read/write mechanism

- `read()`, `write()`, `sendmsg()`, etc.

Programming is often easier with a functional interface

- To make **distributed** computing look more like **centralized** computing

Many components are shared across systems

- e.g., network send/receive; parameter passing, etc. No need to implement from scratch

Goal of RPC

- It should appear to the programmer that a normal call is taking place

Idea: build RPC atop the socket interface

- Hide the construction of messages and the remote invocation logic from the developers

RPC: remote procedure call

Allow a procedure to execute in another address space without coding the details of the remote interaction

RPC history

- The idea goes back to 1976
- Sun's RPC: the first popular implementation on Unix
 - Used as the basis for NFS
- Many modern RPC frameworks: gRPC, Thrift, etc.
- RMI (Remote Method Invocation)
 - Object-oriented version of RPC, e.g., in Java¹

¹ RFC 707, J. White, "A High-Level Framework for Network-Based Resource Sharing", 1976

Example of RPC

```
procedure SYNC_CLOCK ()  
  t ← GET_TIME (SECONDS)  
  SET_CLOCK (t)
```

```
procedure GET_TIME (units)  
  time ← CLOCK  
  time ← CONVERT_TO_UNITS (time, units)  
  return time
```

Suppose the client wants to synchronize its clock with a time server

Assumption

- Only the **server** has an accurate clock and the implementation of `GET_TIME`

How can the client call the server's `GET_TIME`?

Call GET_TIME in the single-machine case

```
procedure SYNC_CLOCK ()  
  t ← GET_TIME (SECONDS)  
  SET_CLOCK (t)
```

```
procedure GET_TIME (units)  
  time ← CLOCK  
  time ← CONVERT_TO_UNITS (time, units)  
  return time
```



Client

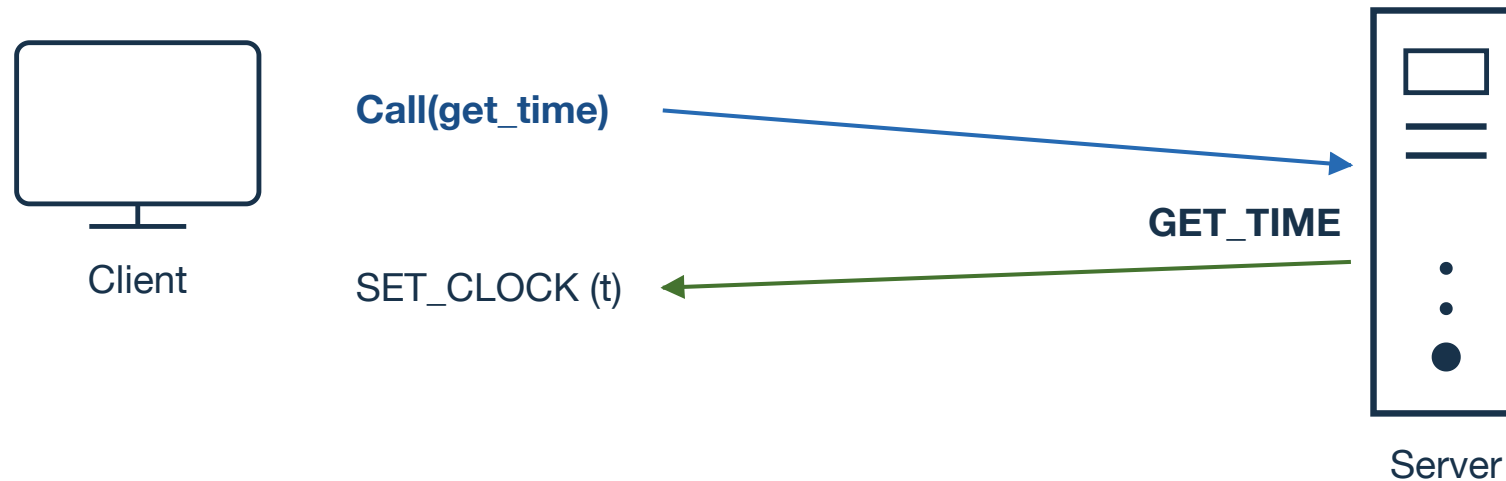
GET_TIME

SET_CLOCK (t)

Single-machine call vs. distributed call (RPC)

```
procedure SYNC_CLOCK ()  
  t ← GET_TIME (SECONDS)  
  SET_CLOCK (t)
```

```
procedure GET_TIME (units)  
  time ← CLOCK  
  time ← CONVERT_TO_UNITS (time, units)  
  return time
```



Handwritten code: client

Without RPC, the `GET_TIME` call becomes a socket `WRITE` and `READ`

```
procedure SYNC_CLOCK ()  
  sock ← CONNECT (TimeServiceAddr)  
  WRITE (sock, {"Get time", CONVERT2EXTERNAL (SECONDS)})  
  response ← READ (sock)  
  t ← CONVERT2INTERNAL (response)  
  SET_CLOCK (t)
```

Handwritten code: server

The server runs `TIME_SERVICE` forever: `READ` a request, run `GET_TIME`, `WRITE` the response

```
procedure TIME_SERVICE ()
  listener ← LISTEN (TimeServicePort)
  sock ← ACCEPT (listener)
  do forever
    request ← READ (sock)
    opcode ← GET_OPCODE (request)
    unit ← CONVERT2INTERNAL (GET_ARGUMENT (request))
    if opcode = "Get time" and (unit = SECONDS or unit = MINUTES) then
      time ← CONVERT_TO_UNITS (CLOCK, unit)
      response ← {"OK", CONVERT2EXTERNAL (time)}
    else
      response ← {"Bad request"}
  WRITE (sock, response)
```

The code only shows the socket calls. A real implementation must consider much more

- e.g., `fault tolerance` (lost messages, server crashes), `performance` (concurrent requests), etc.

RPC simplifies the implementation of remote calls

Abstract away the common parts with a **stub**: the marked lines are provided by the RPC stub

```
procedure SYNC_CLOCK ()  
    sock ← CONNECT (TimeServiceAddr)  
    WRITE (sock, {"Get time", CONVERT2EXTERNAL (SECONDS)})  
    response ← READ (sock)  
    t ← CONVERT2INTERNAL (response)  
    SET_CLOCK (t)
```

RPC simplifies the implementation of remote calls

Abstract away the common parts with a **stub**: the marked lines are provided by the RPC stub

```
procedure TIME_SERVICE ()  
  listener ← LISTEN (TimeServicePort)  
  sock ← ACCEPT (listener)  
  do forever  
    request ← READ (sock)  
    opcode ← GET_OPCODE (request)  
    unit ← CONVERT2INTERNAL (GET_ARGUMENT (request))  
    if opcode = "Get time" and (unit = SECONDS or unit = MINUTES) then  
      time ← CONVERT_TO_UNITS (CLOCK, unit)  
      response ← {"OK", CONVERT2EXTERNAL (time)}  
    else  
      response ← {"Bad request"}  
    WRITE (sock, response)
```

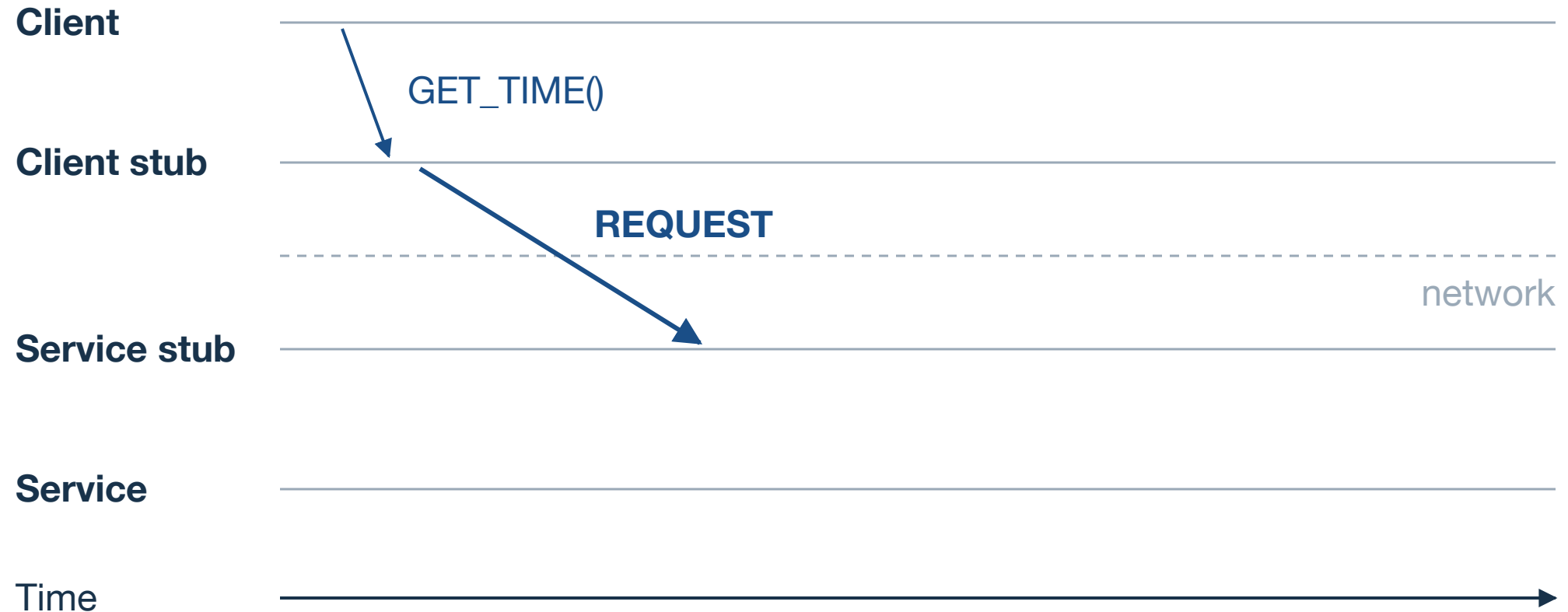

RPC: a complete calling process

1



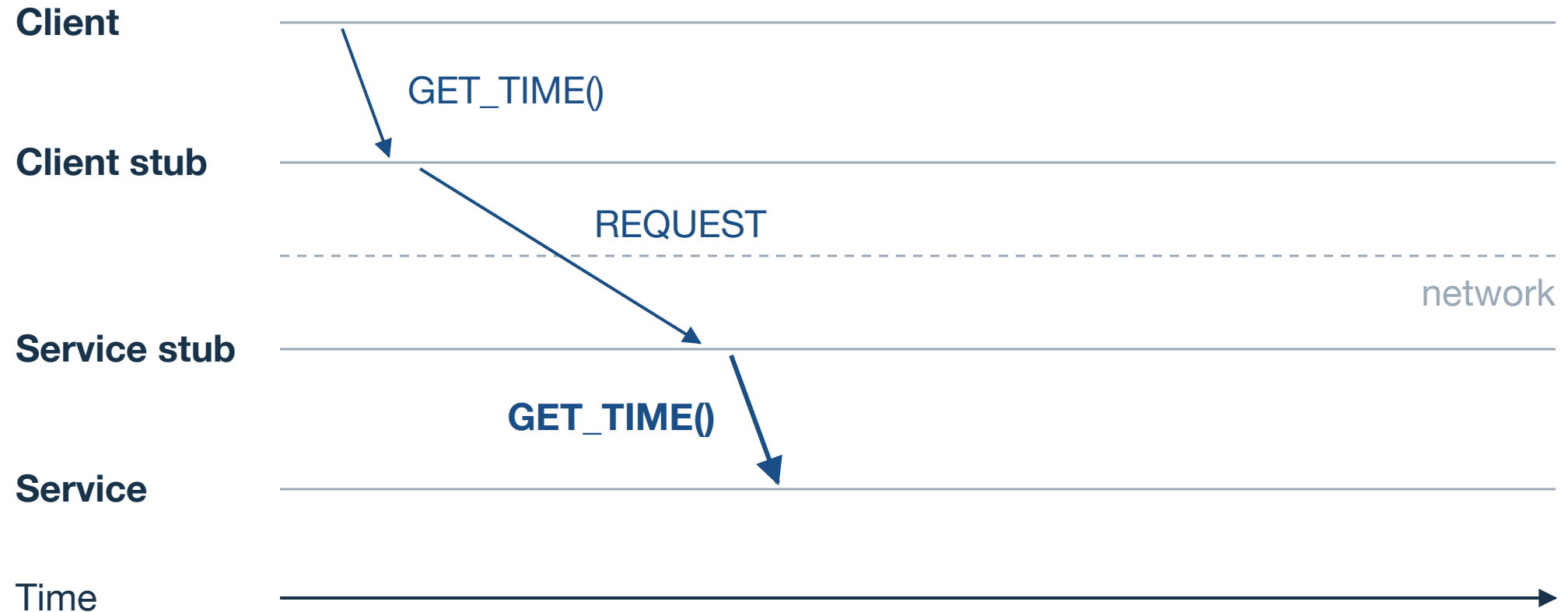
RPC: a complete calling process

1



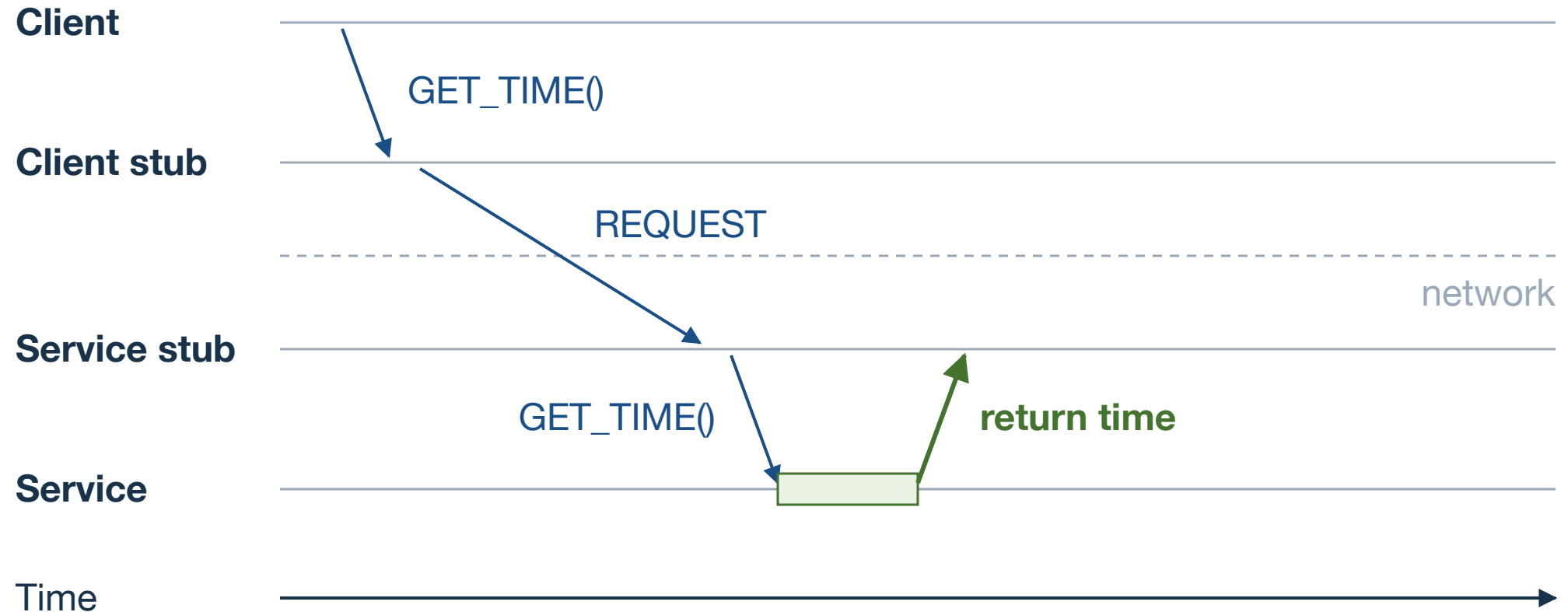
RPC: a complete calling process

1



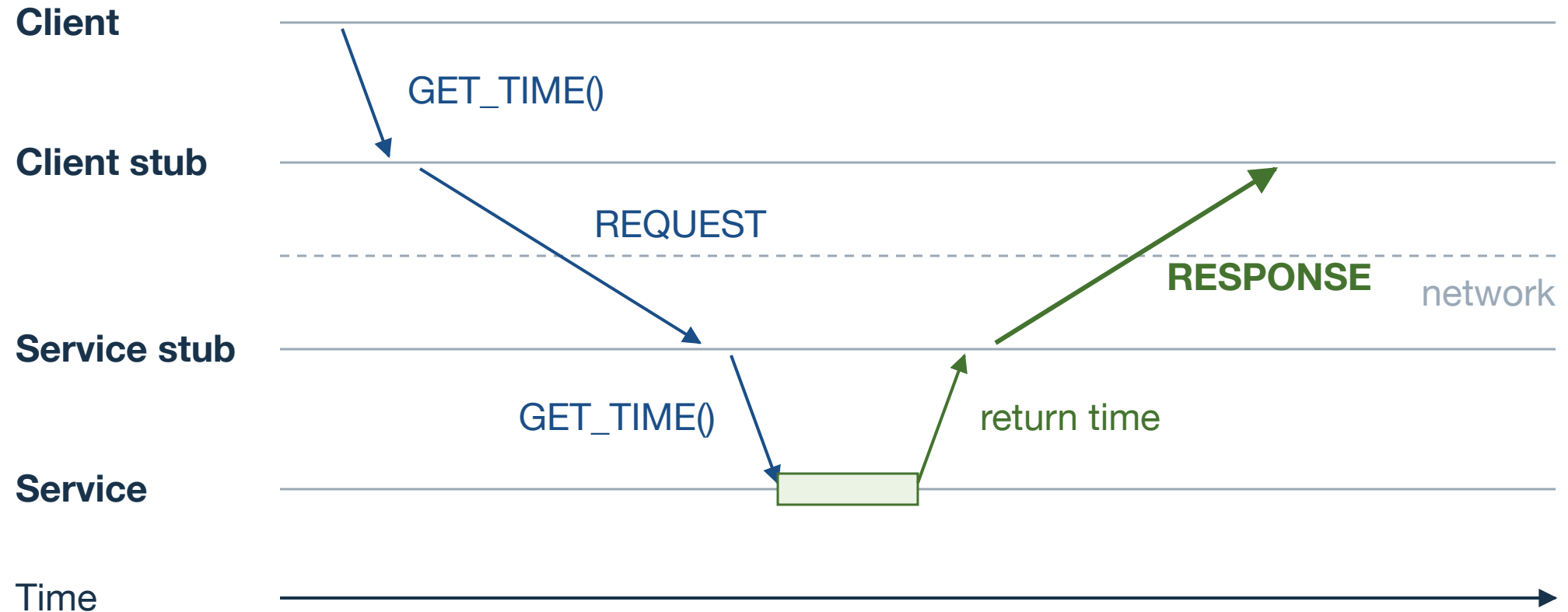
RPC: a complete calling process

1



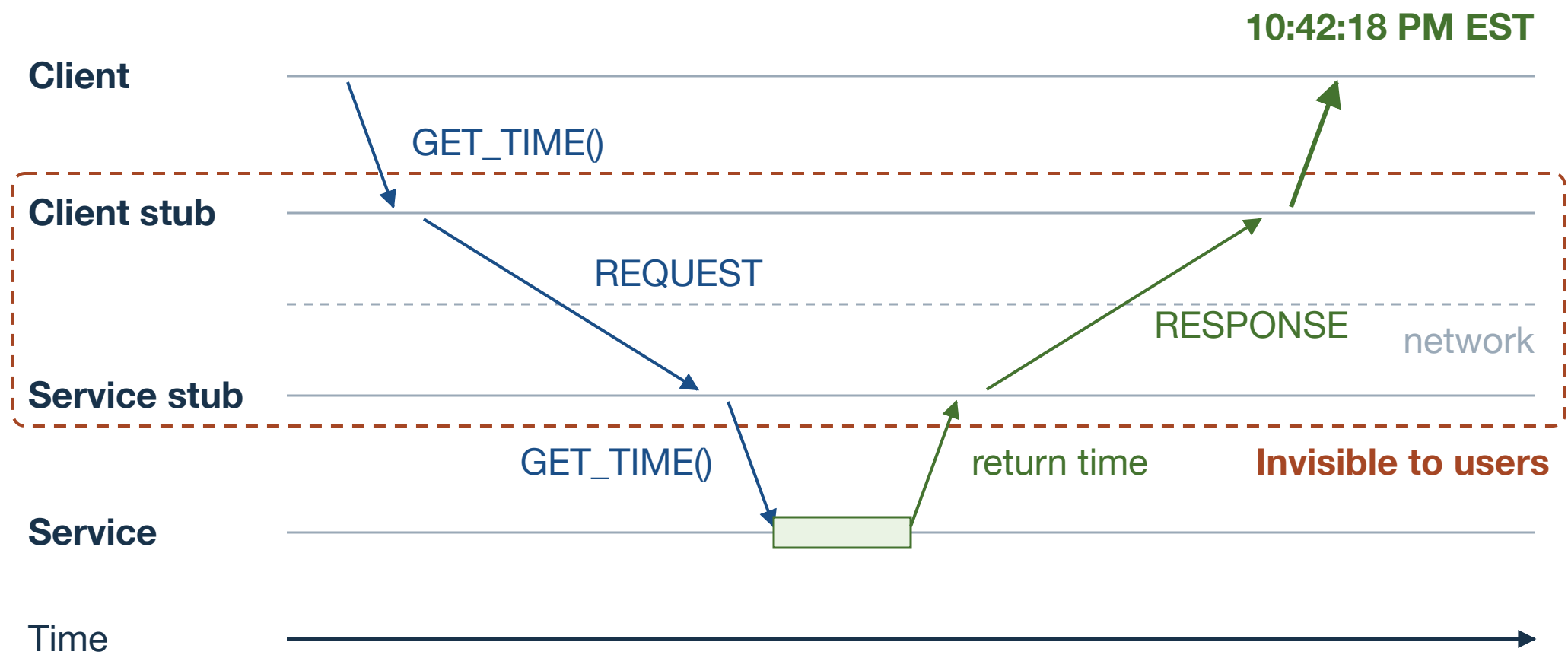
RPC: a complete calling process

1



RPC: a complete calling process

1



¹ Figure adapted from Saltzer & Kaashoek, Principles of Computer System Design

RPC stub

Client stub

- Put the arguments into a request
- Send the request to the server
- Wait for a response

Service stub

- Wait for a message
- Get the parameters from the request
- Call a procedure according to the parameters (e.g., `GET_TIME`)
- Put the result into a response
- Send the response to the client

Stub: hide communication details from upper-level code, so that upper-level code does not change

Client program using RPC

The caller: code is **not changed** compared with the single-machine case

```
procedure SYNC_CLOCK ()  
  t ← GET_TIME (SECONDS)  
  SET_CLOCK (t)
```

The **stub** of the client

```
procedure GET_TIME (units)  
  sock ← CONNECT (TimeServiceAddr)  
  WRITE (sock, {"Get time", CONVERT2EXTERNAL (units)})  
  response ← READ (sock)  
  if GET_RETCODE (response) != "OK"  
    HANDLE_ERROR (response)  
  else  
    return CONVERT2INTERNAL (GET_ARGUMENT (response))
```


Server program using RPC

The implementation: this code is **not changed**

```
procedure GET_TIME (units)
  time ← CLOCK
  time ← CONVERT_TO_UNITS (time, units)
  return time
```

Server program using RPC

The **stub** of the server

```
procedure TIME_SERVICE ()
  listener ← LISTEN (TimeServicePort)
  do forever
    sock ← ACCEPT (listener)
    request ← READ (sock)
    opcode ← GET_OPCODE (request)
    arg ← CONVERT2INTERNAL (GET_ARGUMENT (request))
    if opcode = "Get time" and (arg = SECONDS or arg = MINUTES) then
      retval ← GET_TIME (arg)
      response ← {"OK", CONVERT2EXTERNAL (retval)}
    else
      response ← {"Bad request"}
  WRITE (sock, response)
```

Question: what is inside a message?

Client stub

```
WRITE (sock, {"Get time", CONVERT2EXTERNAL (units)})  
response ← READ (sock)
```

Server stub

```
request ← READ (sock)  
...  
WRITE (sock, response)
```

Question: what is inside a message?

A message may contain

- Service ID (e.g., function ID)
- Service parameters (e.g., function parameters)
- Encoded with marshal / unmarshal

RPC request message

RPC request

- **xid**: X is short for “transaction”; the client remembers the xid of each call and dispatches replies by xid
- call/reply
- RPC version
- **program #**: the server dispatches by prog# and proc#
- program version
- **procedure #**
- auth stuff
- arguments

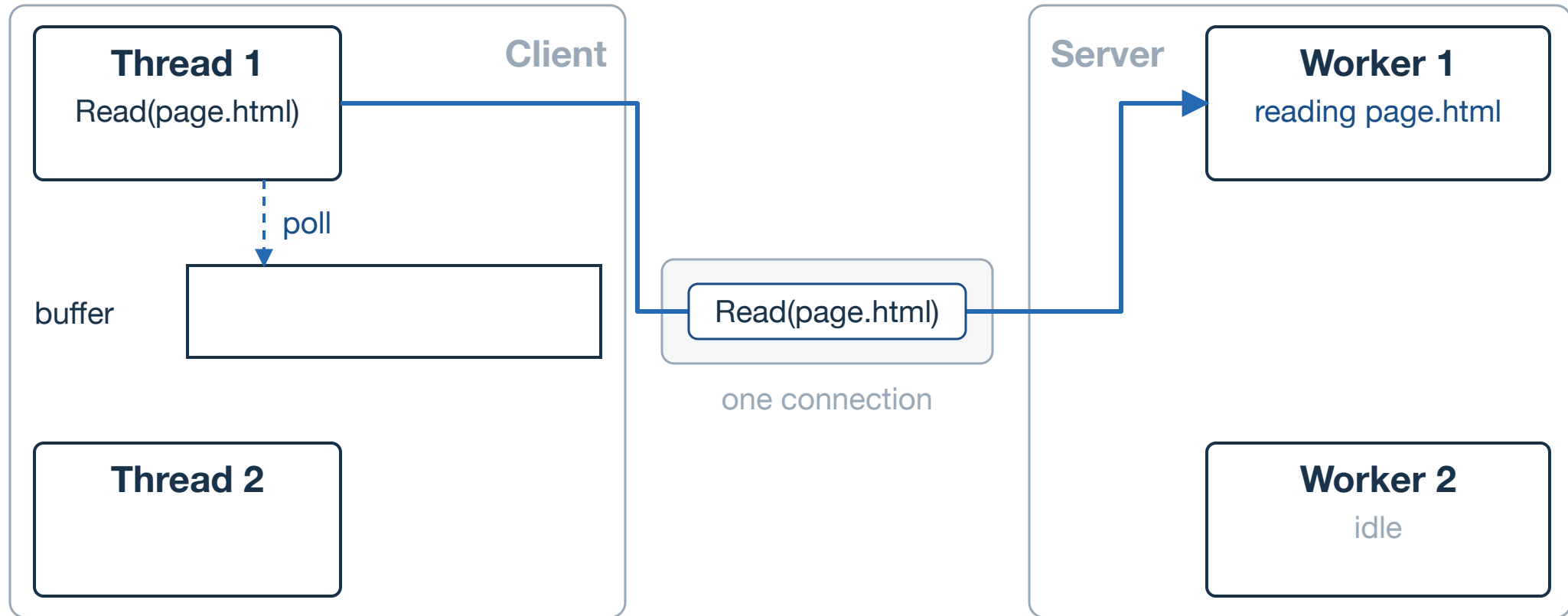
RPC reply message

RPC reply

- xid
- call/reply
- accepted? (Yes, or No due to bad RPC version, auth failure, etc.)
- auth stuff
- success? (Yes, or No due to bad prog/proc #, etc.)
- results

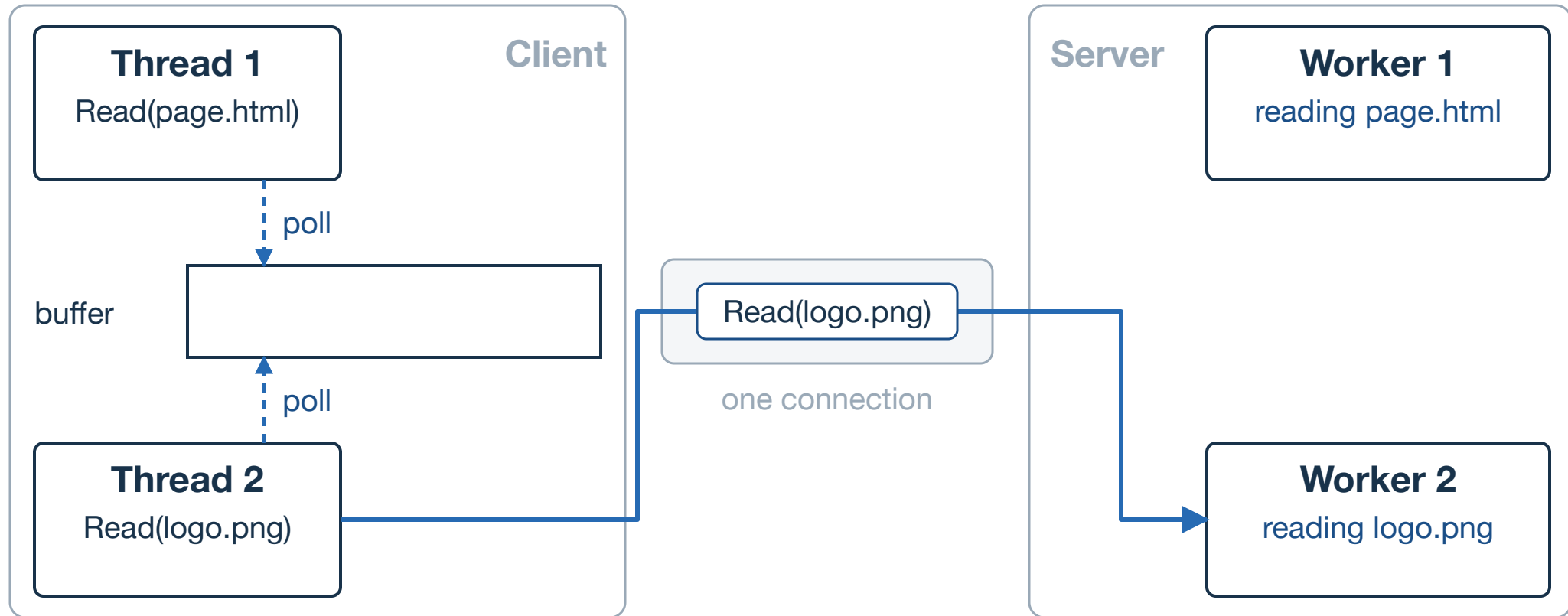
Why does a request need an xid?

Client threads share **one connection**. The server handles requests with **many threads**



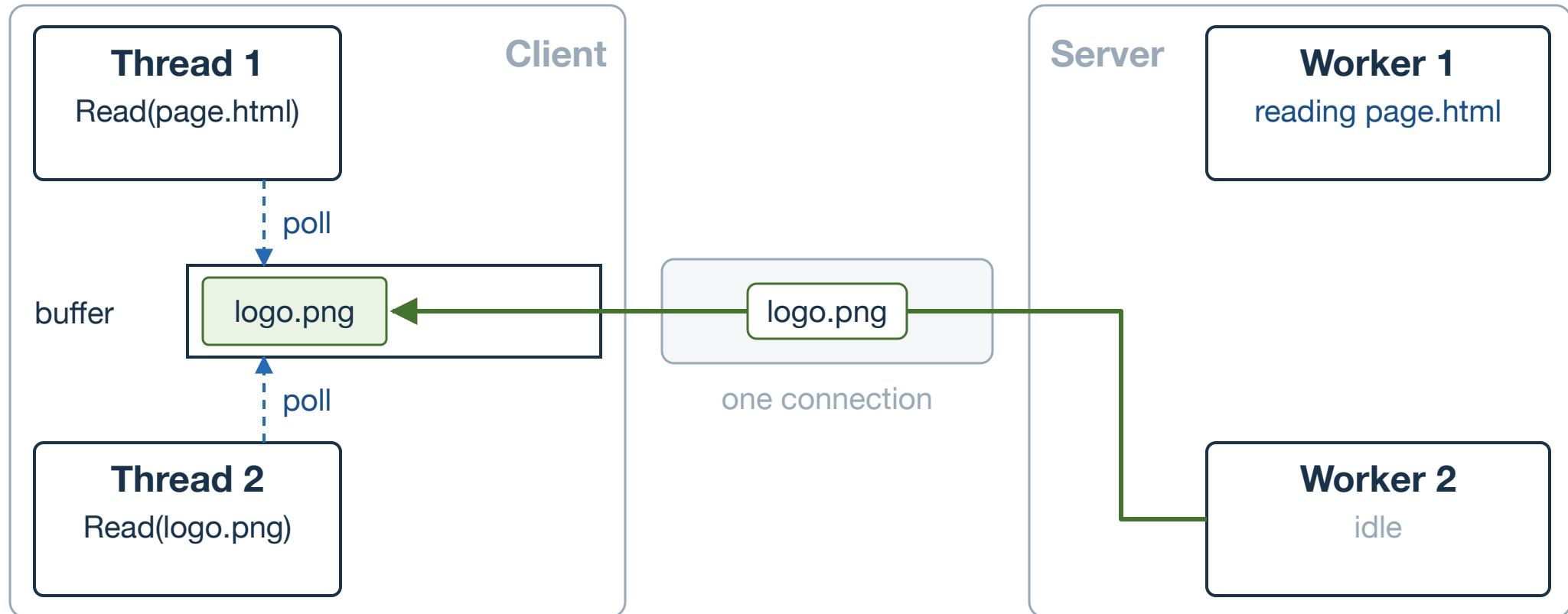
Why does a request need an xid?

Client threads share **one connection**. The server handles requests with **many threads**



Why does a request need an xid?

Client threads share **one connection**. The server handles requests with **many threads**



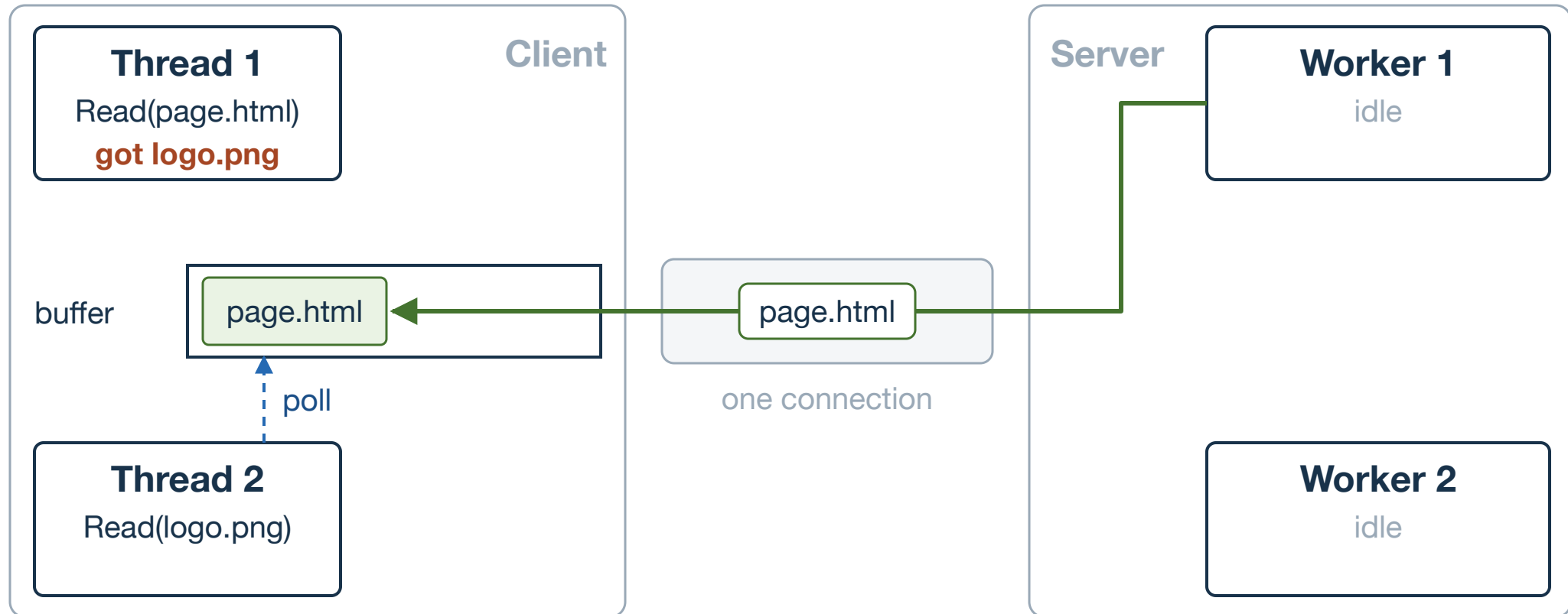
Why does a request need an xid?

Client threads share **one connection**. The server handles requests with **many threads**



Why does a request need an xid?

Client threads share **one connection**. The server handles requests with **many threads**



Why does a request need an xid?

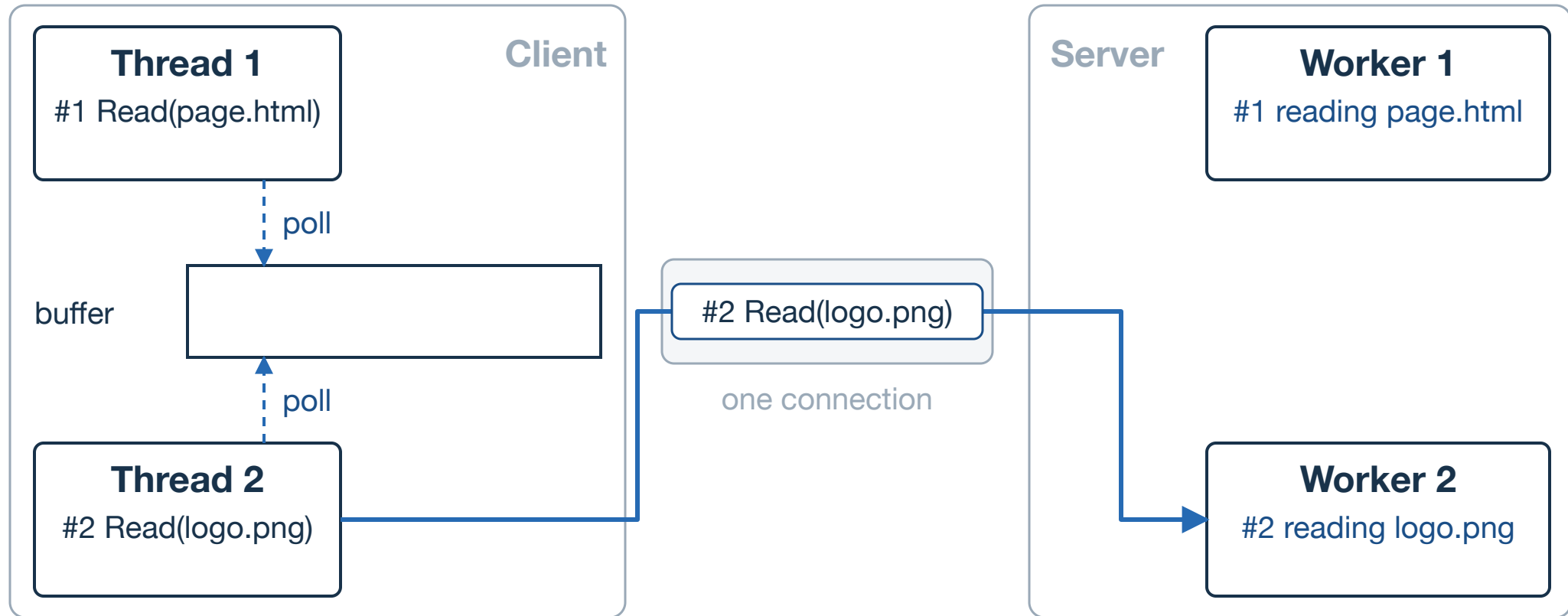
Client threads share **one connection**. The server handles requests with **many threads**



Without xid, a polling thread takes **any** reply in the buffer

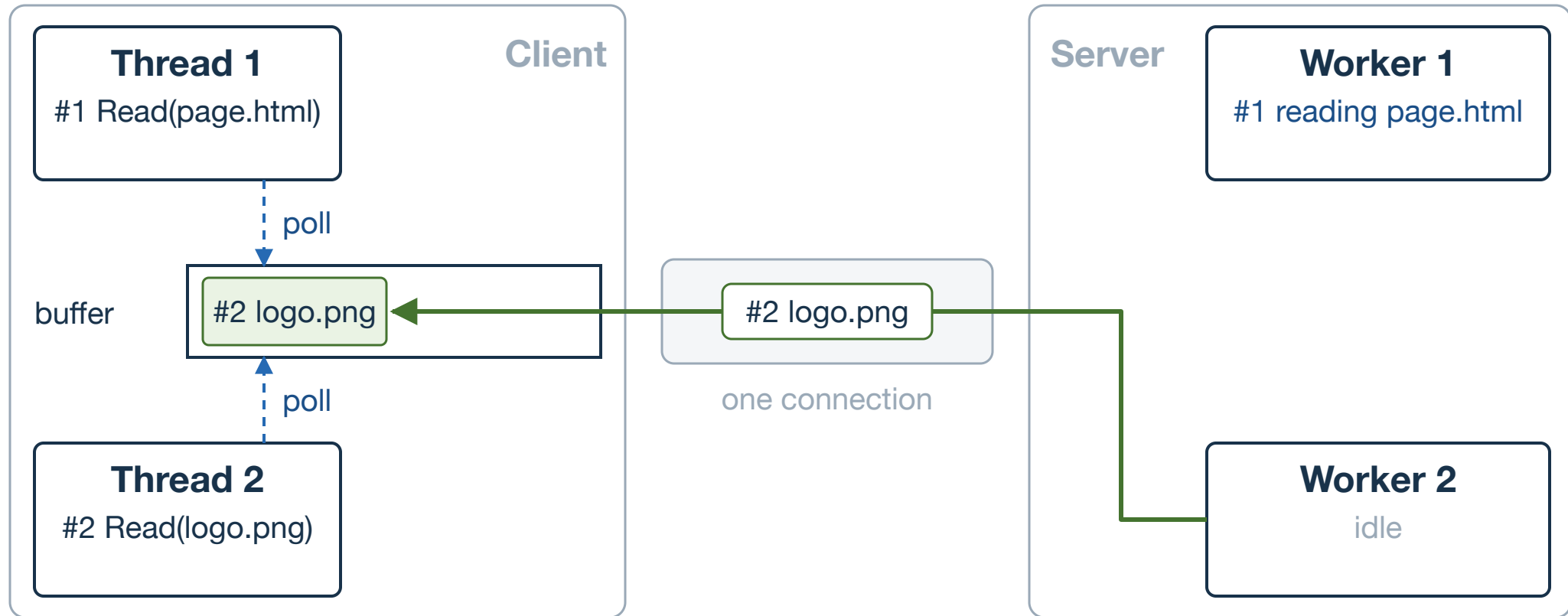
Why does a request need an xid?

A request and its reply carry the **same xid**



Why does a request need an xid?

A request and its reply carry the **same xid**



Why does a request need an xid?

A request and its reply carry the **same xid**



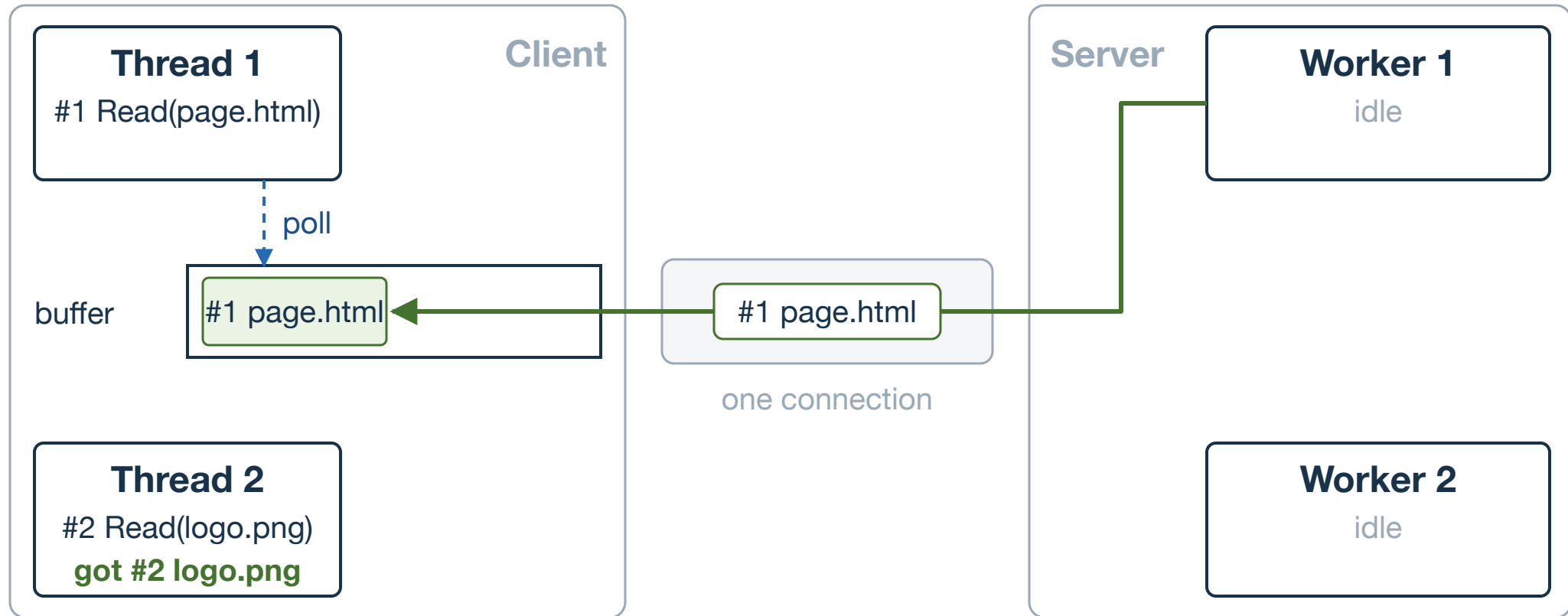
Why does a request need an xid?

A request and its reply carry the **same xid**



Why does a request need an xid?

A request and its reply carry the **same xid**



Why does a request need an xid?

A request and its reply carry the **same xid**



A polling thread takes a reply only if the **xid matches**

Passing data between client and server is more complicated

Parameter passing

Pass by **value**?

- Easy: just copy data to the network message

Pass by **reference**?

- Makes no sense without shared memory

Need a conversion between data used in a program and data that can be transferred through the network

- The client converts the data structure into a **pointerless** representation
- The client transmits the data to the server
- The server reconstructs the structure with local pointers

Parameter passing is challenging across machines

Distributed systems have the **incompatibility** problem

- which does not exist on a single machine

For example, a remote machine may have **different**

- byte ordering
- sizes of integers and other types
- floating-point representations
- character sets, e.g., "中" is **D6 D0** in GBK but **E4 B8 AD** in UTF-8
- alignment requirements
- etc.¹

Represent 0x89abcdef

Big endian, e.g., Power

89	AB	CD	EF
0	1	2	3

Little endian, e.g., x86

EF	CD	AB	89
0	1	2	3

¹ gRPC: a Protobuf `string` is always UTF-8 on the wire, see [Protobuf docs](#)

Parameter passing is more challenging

Applications change over time

- What would happen if a server **upgrades** while the client stays unchanged?
- E.g., the server upgrades and adds a priority field to its message; what would happen if the client does not change?

Evolvability: we should build systems that are easy to adapt to changes

- **Backward compatibility:** newer code can read data that was written by older code
- **Forward compatibility:** older code can read data that was written by newer code

```
old: struct Msg { int id; }  
new: struct Msg { int id; int priority; }
```

```
Backward: old writes {id: 7}          -> new reads {id: 7, priority: 0}
```

```
Forward:  new writes {id: 7, priority: 1} -> old reads {id: 7}
```

Representing data: encoding

Need a standard **encoding** to enable communication between heterogeneous systems and different versions of software

- Sun's RPC uses XDR (eXternal Data Representation)
- ASN.1 (ISO Abstract Syntax Notation)
- JSON
- Google Protocol Buffers
- W3C XML Schema Language



Why not use language-specific formats?

Many languages have **built-in support** for encoding in-memory objects

- Java's `java.io.Serializable`
- Python's `pickle`



Drawbacks

- The encoding is tied to a particular programming language
 - E.g., it is challenging to use a Java client to call a Python server
- No versioning → no forward and backward compatibility

System requirements for encoding/decoding

Transfer objects through the network

- Correctly encode an object into a byte stream and decode it back

Compatibility

- Support multiple languages and multiple versions of a program

Efficiency

- Reduce the traffic transferred over the network
- Network bandwidth is a scarce resource

Standardized encoding: JSON, XML, etc.

Independent of a specific programming language

Textual formats

- JSON
- XML/YAML/TOML
- CSV

Logical client request format → JSON representation

- xid, call/reply, RPC version, program #, program version, procedure #, auth stuff, arguments

```
{ "xid": 12, "call": true,  
  "rpc_version": 73,  
  ...  
}
```

Standardized encoding: JSON, XML, etc.

Independent of a specific programming language

Textual formats

- JSON
- XML
- CSV

Benefits

- Human-readable: easy to debug

Standardized encoding: JSON, XML, etc.

Independent of a specific programming language; **textual formats**: JSON, XML, CSV

```
{ "xid": 12, "call": true,  
  "rpc_version": 73,  
  ...  
}
```

Drawbacks

1. Ambiguity around the encoding of numbers
2. How to support **binary strings**?
 - Programmers have to **re-encode** the string as Base64, etc.
3. **Verbose**: uses more bytes to store the data, e.g., tags like `<xx> </xx>` in XML

Binary formats

Encode the data using **binary encoding**

- Pros: more compact, more accurate, faster to parse and store
- Cons: less human-readable
 - Not a problem when the data is only used internally
 - E.g., RPC message metadata

Typical examples: Thrift & Protocol Buffers

- Thrift: by Facebook, now an Apache project; Protocol Buffers: by Google

```
{ "userName": "Martin", "favoriteNumber": 1337, "interests": ["daydreaming", "hacking"] }
```

The JSON representation takes 81 bytes in total (without spaces)

- Binary formats can reduce it to **34 bytes or even less**

Binary formats: schema

Both Thrift and Protocol Buffers require a **schema** for any data that is encoded

- Benefits: no need to encode things such as `userName` in the encoded data

Thrift interface definition language (IDL) vs. Protocol Buffers IDL

```
// Thrift IDL                                // Protocol Buffers IDL
struct Person {                               message Person {
  1: required string      userName,           required string user_name      = 1;
  2: optional i64         favoriteNumber,     optional int64  favorite_number = 2;
  3: optional list<string> interests          repeated string interests      = 3;
}
```

JSON representation (81 bytes without spaces)

```
{ "userName": "Martin", "favoriteNumber": 1337, "interests": ["daydreaming", "hacking"] }
```

The Binary Protocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order

1: string userName

"Martin"

2: i64 favoriteNumber

1337

3: list<string> interests

"daydreaming"

"hacking"

The BinaryProtocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order

1: string userName

"Martin"

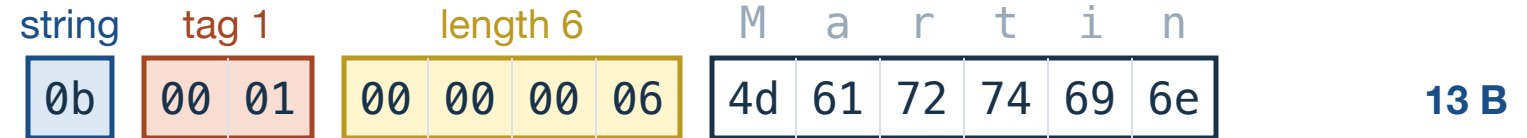
2: i64 favoriteNumber

1337

3: list<string> interests

"daydreaming"

"hacking"



The BinaryProtocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order

1: string userName

"Martin"

string

tag 1

length 6

M a r t i n

0b

00 01

00 00 00 06

4d 61 72 74 69 6e

13 B

2: i64 favoriteNumber

1337

i64

tag 2

1337

0a

00 02

00 00 00 00 00 00 05 39

24 B

3: list<string> interests

"daydreaming"

"hacking"

The BinaryProtocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order

1: string userName

"Martin"

string

tag 1

length 6

M a r t i n

0b

00 01

00 00 00 06

4d 61 72 74 69 6e

13 B

2: i64 favoriteNumber

1337

i64

tag 2

1337

0a

00 02

00 00 00 00 00 00 05 39

24 B

3: list<string> interests

"daydreaming"

"hacking"

list

tag 3

string

2 items

0f

00 03

0b

00 00 00 02

32 B

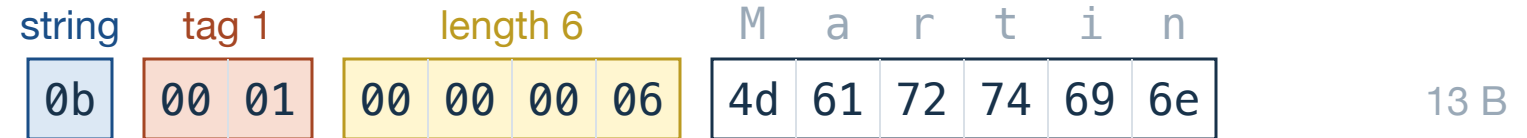
The BinaryProtocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order

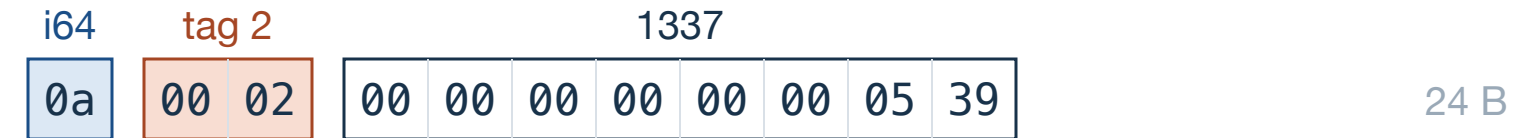
1: string userName

"Martin"



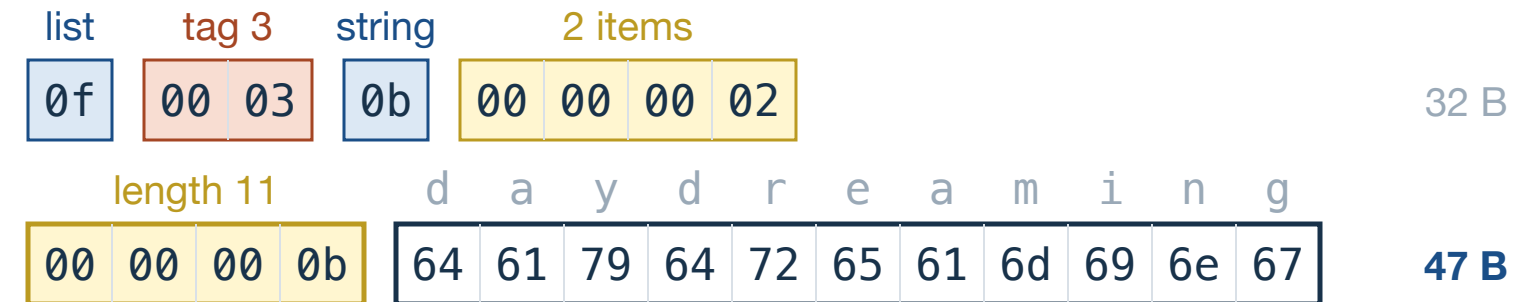
2: i64 favoriteNumber

1337



3: list<string> interests

"daydreaming"



"hacking"

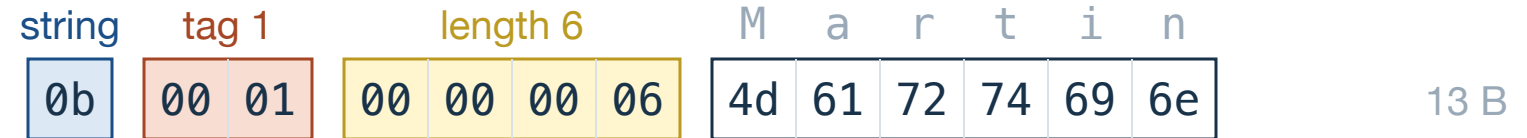
The BinaryProtocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order

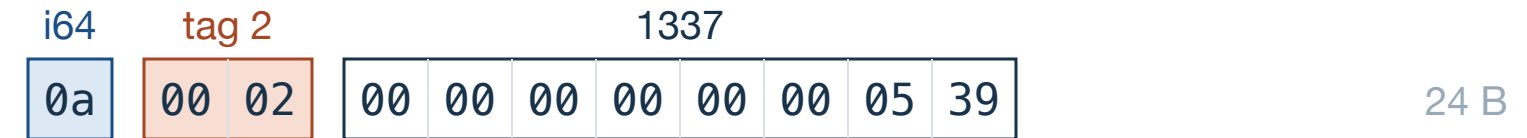
1: string userName

"Martin"



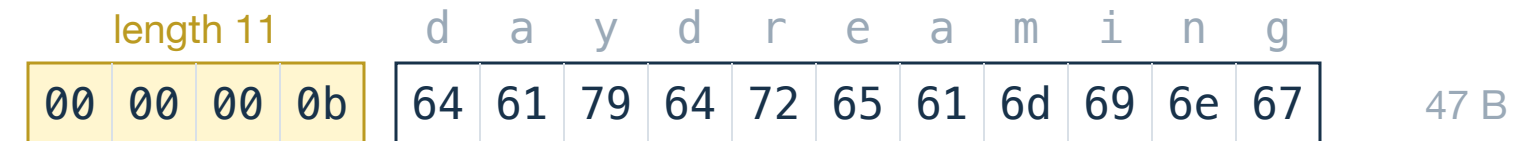
2: i64 favoriteNumber

1337

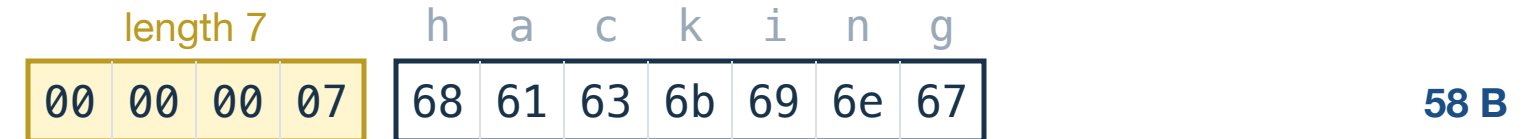


3: list<string> interests

"daydreaming"



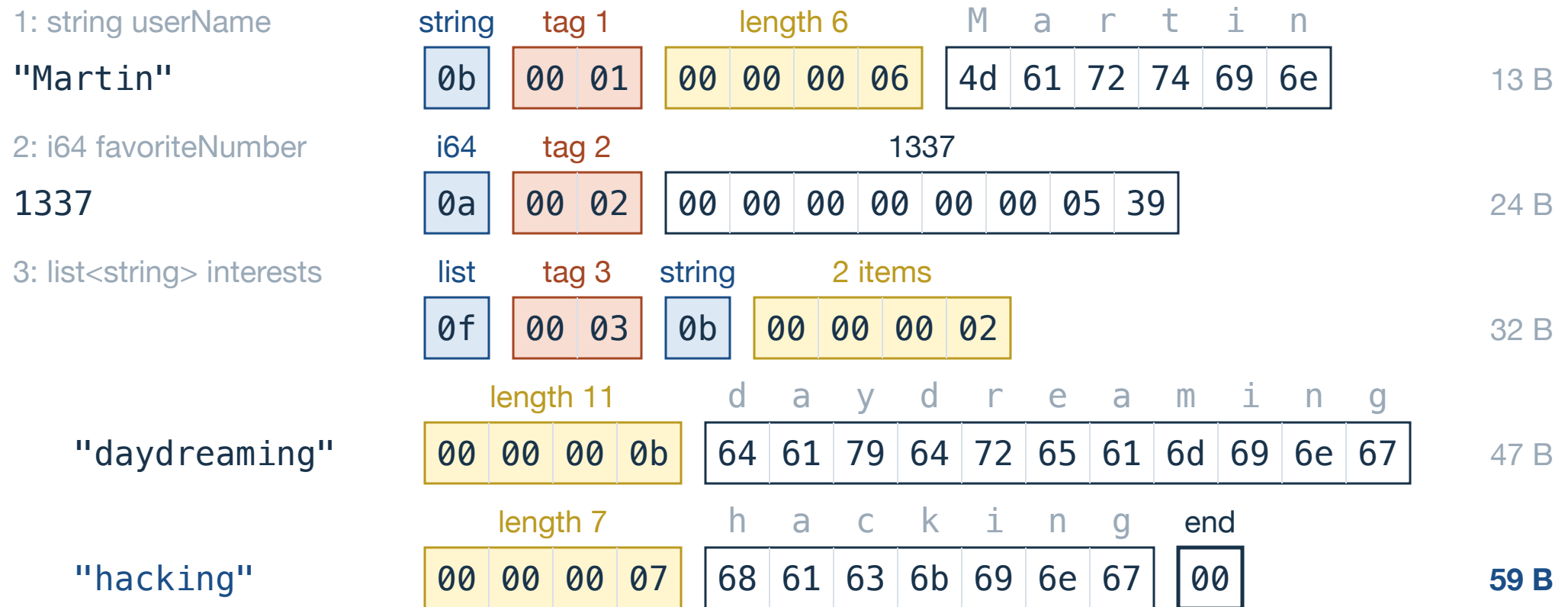
"hacking"



The BinaryProtocol of Thrift

Each field: **type** (1B) + **field tag** (2B) + **length** (string, list, etc.) + data

- Integers use a fixed size and big-endian byte order



Total **59B**: no field names (`userName`, `favoriteNumber`, `interests`)

Being more compact: Thrift CompactProtocol

1. Pack the field tag (delta from the previous field) and type into 1B
2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*

1: string userName

"Martin"

2: i64 favoriteNumber

1337

3: list<string> interests

"daydreaming"

"hacking"

Being more compact: Thrift CompactProtocol

- 1. Pack the field tag (delta from the previous field) and type into 1B
- 2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*

1: string userName

"Martin"

tag +1 string

length 6

M a r t i n

4d617274696e

8 B

2: i64 favoriteNumber

1337

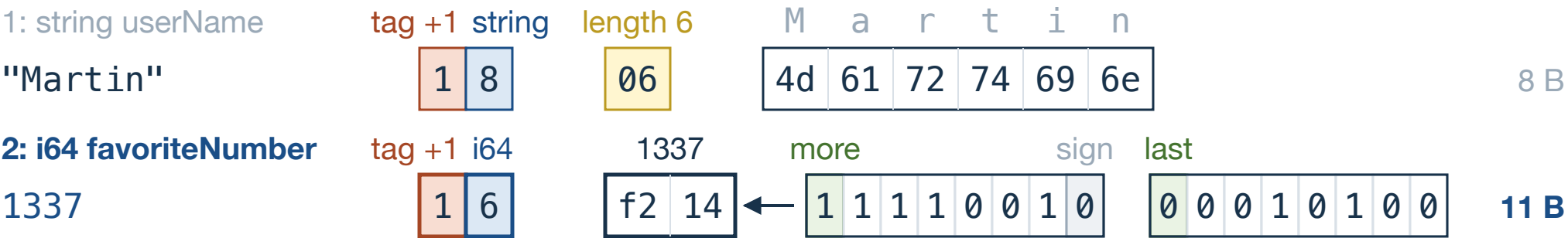
3: list<string> interests

"daydreaming"

"hacking"

Being more compact: Thrift CompactProtocol

- 1. Pack the field tag (delta from the previous field) and type into 1B
- 2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*

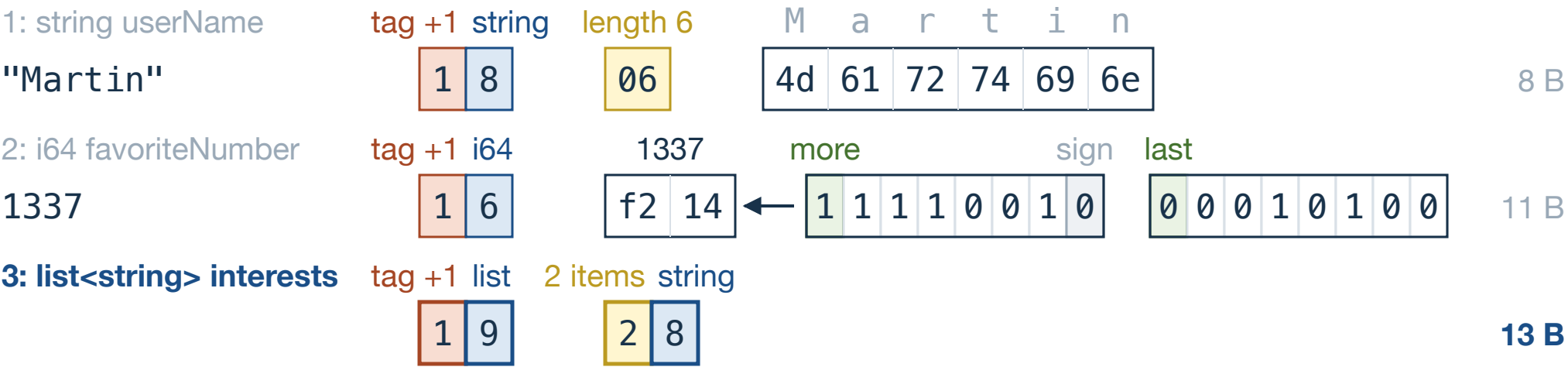


"daydreaming"

"hacking"

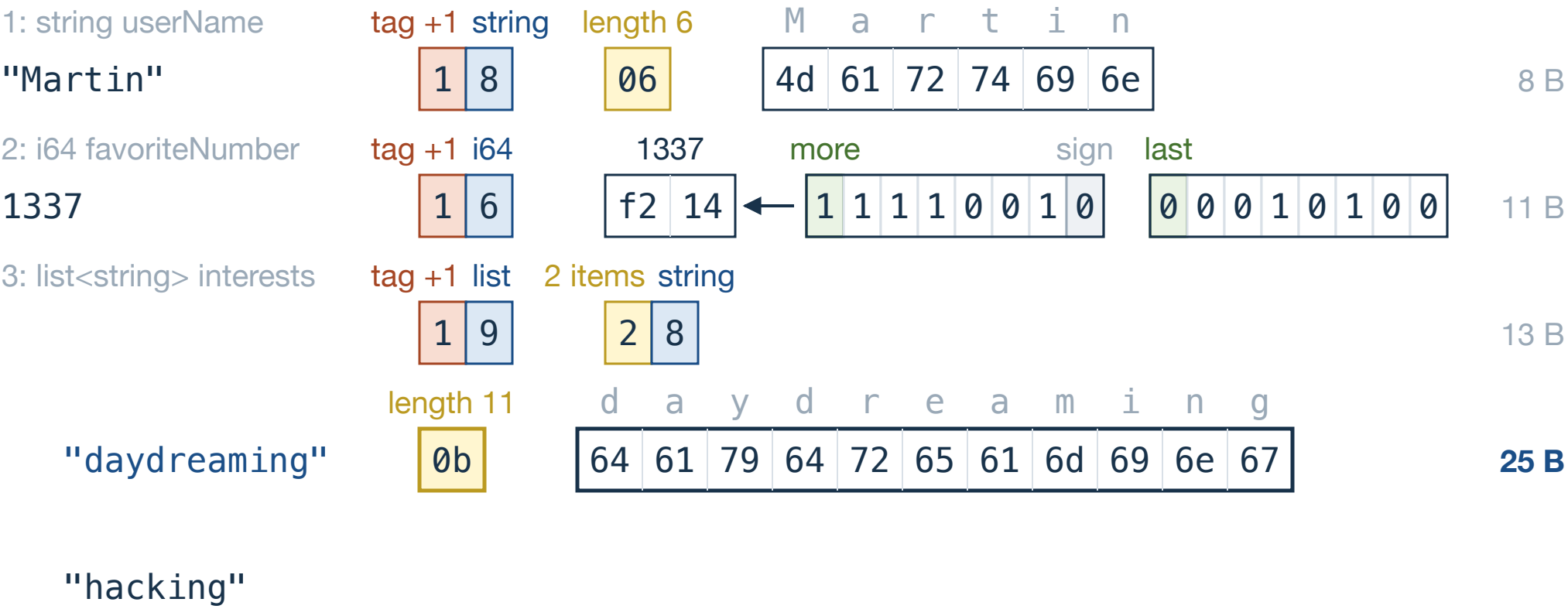
Being more compact: Thrift CompactProtocol

- 1. Pack the field tag (delta from the previous field) and type into 1B
- 2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*



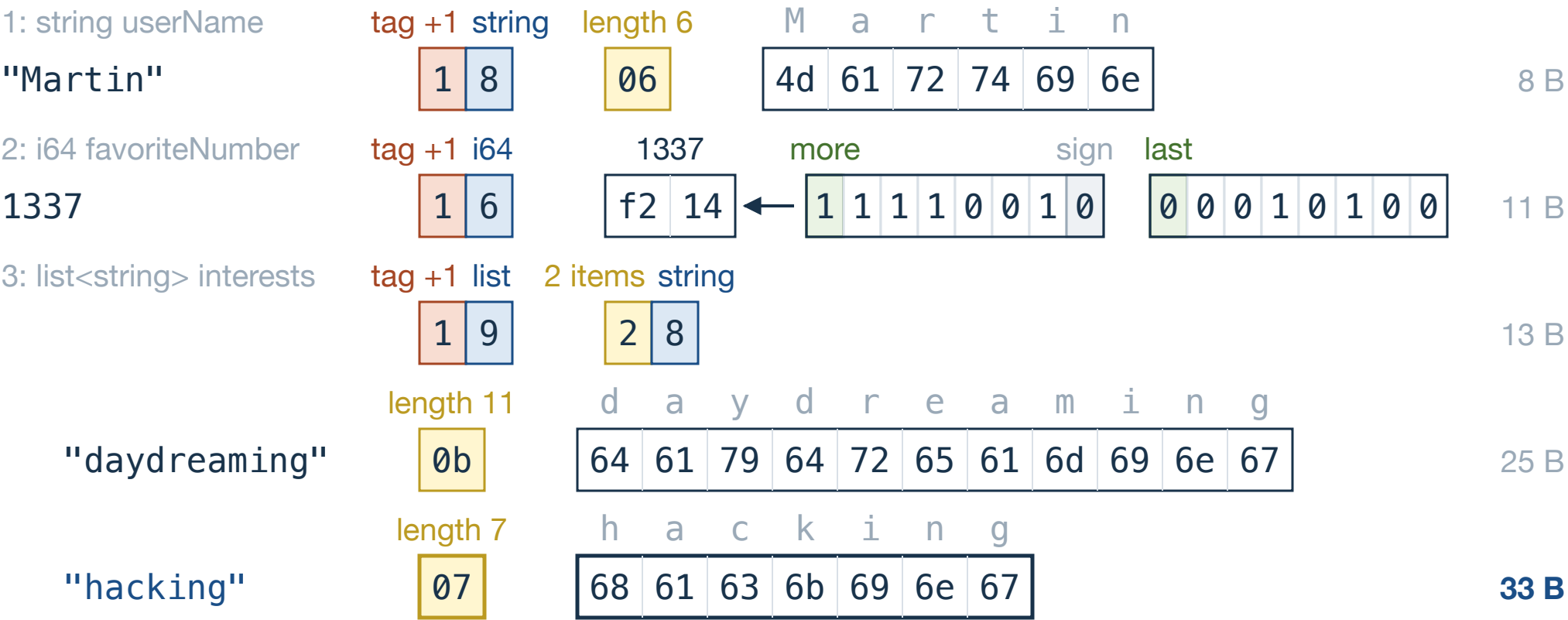
Being more compact: Thrift CompactProtocol

- 1. Pack the field tag (delta from the previous field) and type into 1B
- 2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*



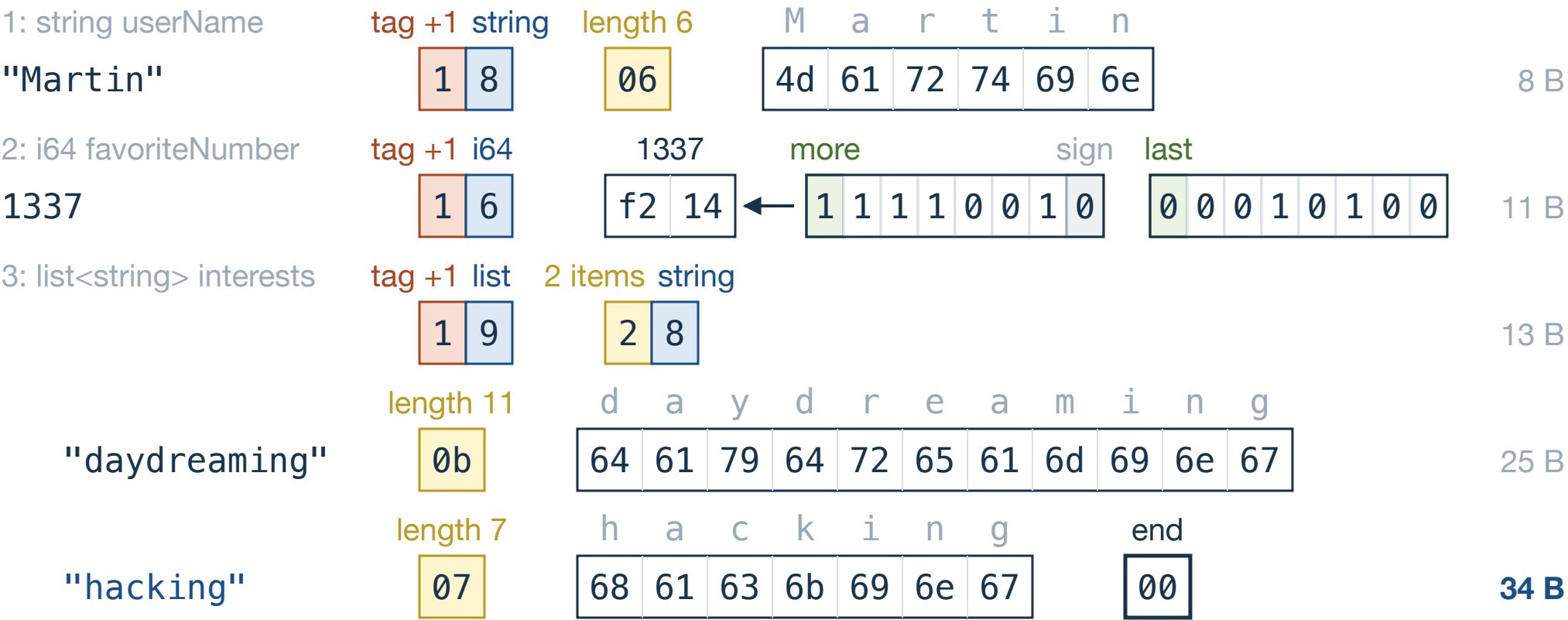
Being more compact: Thrift CompactProtocol

- 1. Pack the field tag (delta from the previous field) and type into 1B
- 2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*



Being more compact: Thrift CompactProtocol

- 1. Pack the field tag (delta from the previous field) and type into 1B
- 2. Variable-length integer: *the top bit of each byte indicates whether there are more bytes*



Now only **34B** (BinaryProtocol: 59B); 1337: from **8B** to **2B**

Schema simplifies supporting compatibility

Key observation: we only need to keep the **field tags** and **field types** compatible

Old schema

```
struct Person {  
  1: string userName  
  2: i64 favoriteNumber  
}
```

=

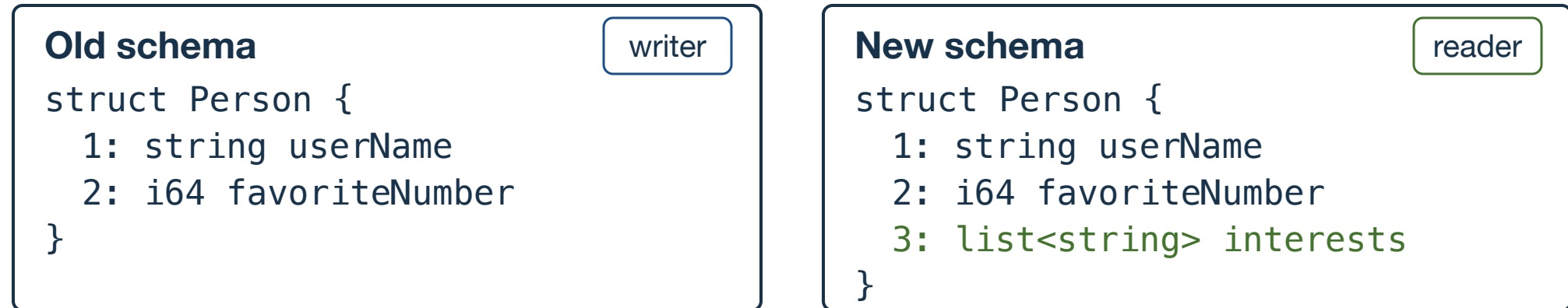
New schema

```
struct Person {  
  1: string userName  
  2: i64 favoriteNumber  
  3: list<string> interests new  
}
```

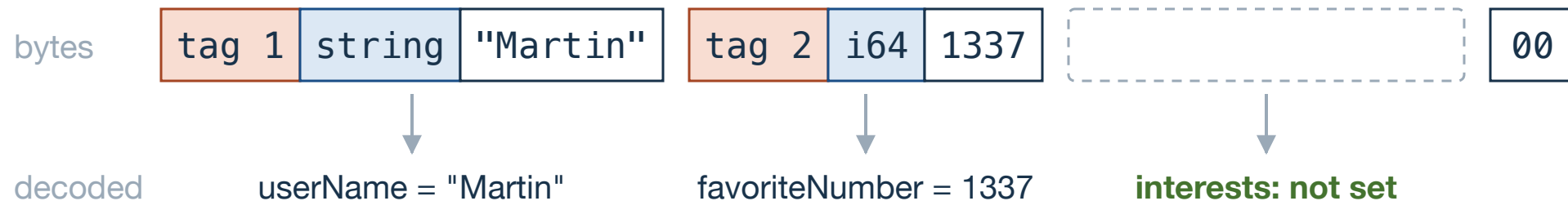
same field tags & types

Schema simplifies supporting compatibility

Key observation: we only need to keep the **field tags** and **field types** compatible

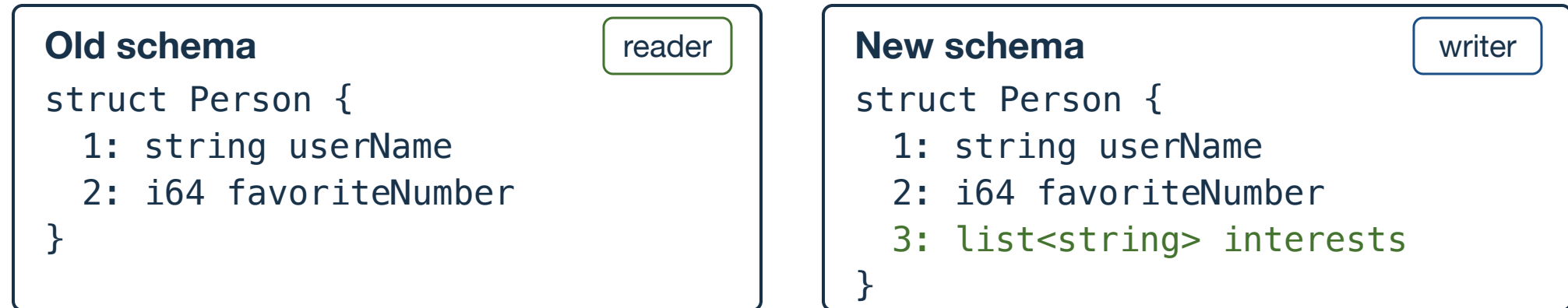


Backward: new code reads old data

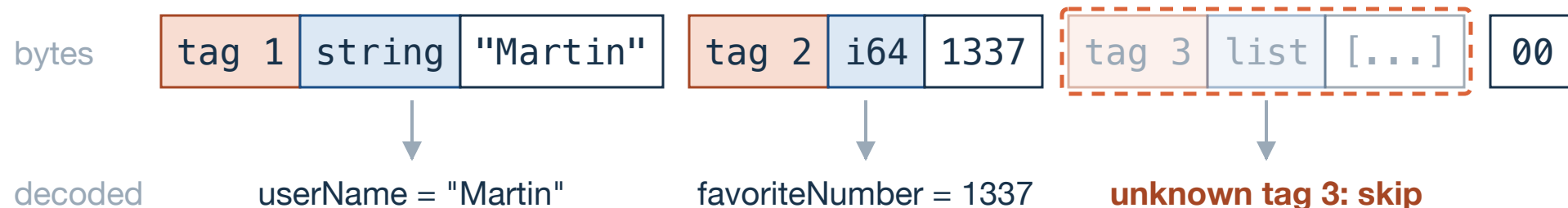


Schema simplifies supporting compatibility

Key observation: we only need to keep the **field tags** and **field types** compatible



Forward: old code reads new data



Old code **skips** an **unknown field tag**; the field type tells how many bytes to skip

Short summary of parameter passing in RPC

Passing data using RPC is non-trivial; it should ensure

- Correctness
- Compactness
- Evolvability

Encoding/decoding

- Convert an object into **an array of bytes** with enough annotation so that the decode procedure can convert it back into an object
- Also known as **marshal / unmarshal** in RPC
- Other names: **serialize / deserialize**

Automatic stub generation

Generate stubs from an **interface specification**

- A tool looks at the argument and return types
- Generates the **marshal** and **unmarshal** code
- Generates the stub procedures

Benefits

- Saves programming (thus fewer errors)
- Ensures agreement on argument types
 - E.g., consistent function IDs

Typically a standard component in an RPC library

Transport protocol of RPC

Stubs also include implementations of sending/receiving messages

- TCP or UDP, which one to use?
- What about new networking features, e.g., RDMA?

Hide the transport protocol from the user

- Benefits: e.g., transparently migrate to a more advanced network

Most RPC systems support several

- Allow the programmer (or end user) to choose at runtime based on their hardware setup or performance requirements

Short summary of encoding & decoding data for RPC

Transfer objects through the network

- Correctly encode an object into a byte stream and decode it back

Compatibility

- Support multiple languages and multiple versions of a program

→ **uniform format + schema**

Efficiency

- Reduce the traffic transferred over the network
- Network bandwidth is a scarce resource

→ **binary format**

Short summary of encoding & decoding data for RPC

Efficiency

- Reduce the traffic transferred over the network
- Network bandwidth is a scarce resource

Question

- Does a binary format always improve the efficiency of RPC?
- E.g., what if the network becomes faster?

Short summary: RPC so far

RPC simplifies programming with an interface similar to a local function call

RPC uses stubs to avoid handling argument **encoding/decoding** and sending/receiving messages, etc.

- Ensure correctness & efficiency

Remaining challenges of RPC

- How to handle failure?
 - As we have mentioned, failure is common, especially in large-scale distributed systems

When RPC meets failure

When RPC meets failures

Local procedure calls do not fail (most of the time)

- If that happens, the entire process dies

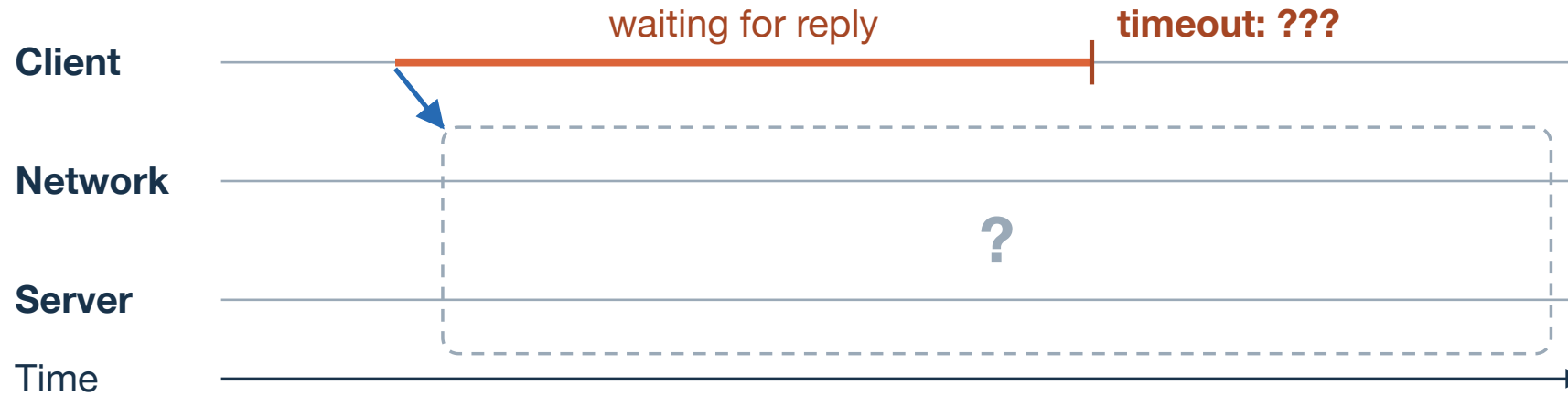
RPC may meet many different failures

- Parameter error → just return
- Server crash → how does the client cope with it?

A crash can happen even if the remote procedure runs correctly!

When RPC meets failures

1



RPC may suffer a **partial failure**

- Not all components fail

A client uses RPC to call a function at a service through the network

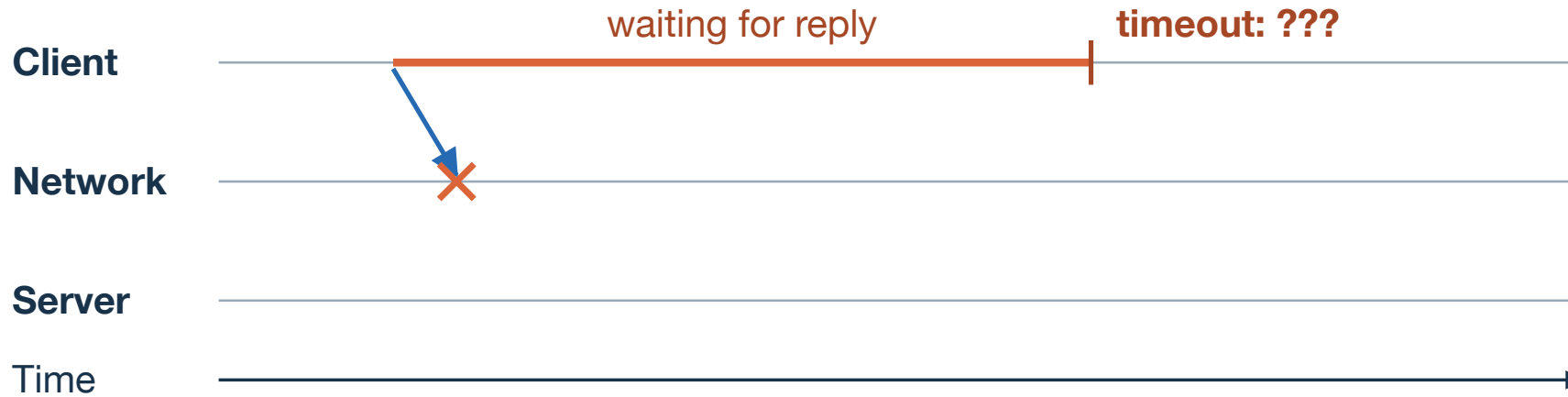
- The client does not get any reply
- What could possibly happen?

¹ Adapted from Kleppmann, Designing Data-Intensive Applications, Figure 8-1

When RPC meets failures

1. Request lost

1



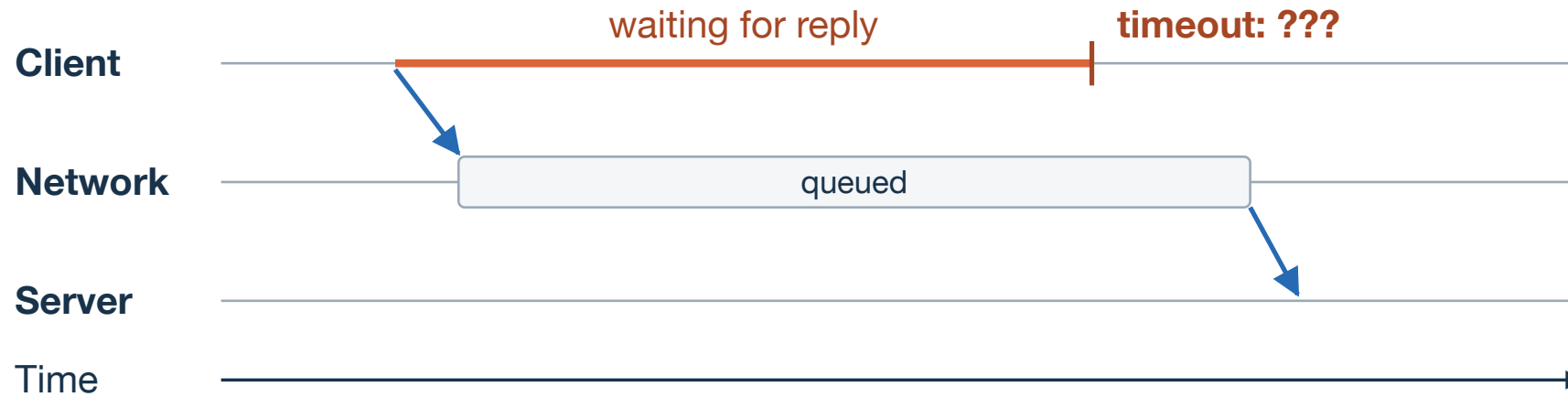
A user sends an RPC but the server **does not reply**; possible reasons:

1. The request may have been **lost** (e.g., someone unplugged a network cable)

When RPC meets failures

2. Request delayed

1



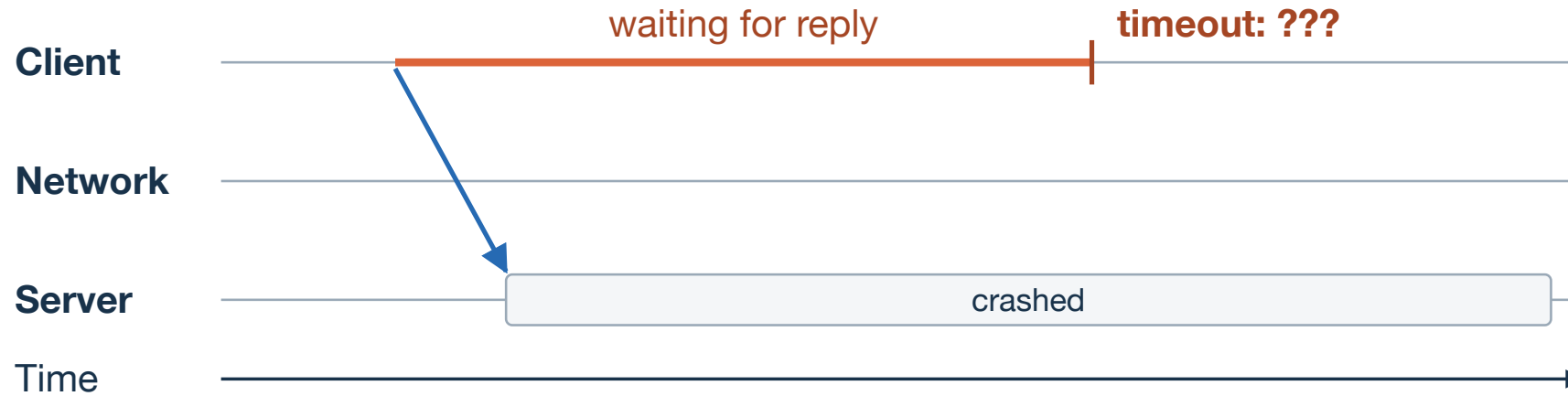
A user sends an RPC but the server **does not reply**; possible reasons:

2. The request may be waiting in a queue and will be delivered **later** (e.g., the network or the recipient is overloaded)

When RPC meets failures

3. Server crashed

1



A user sends an RPC but the server **does not reply**; possible reasons:

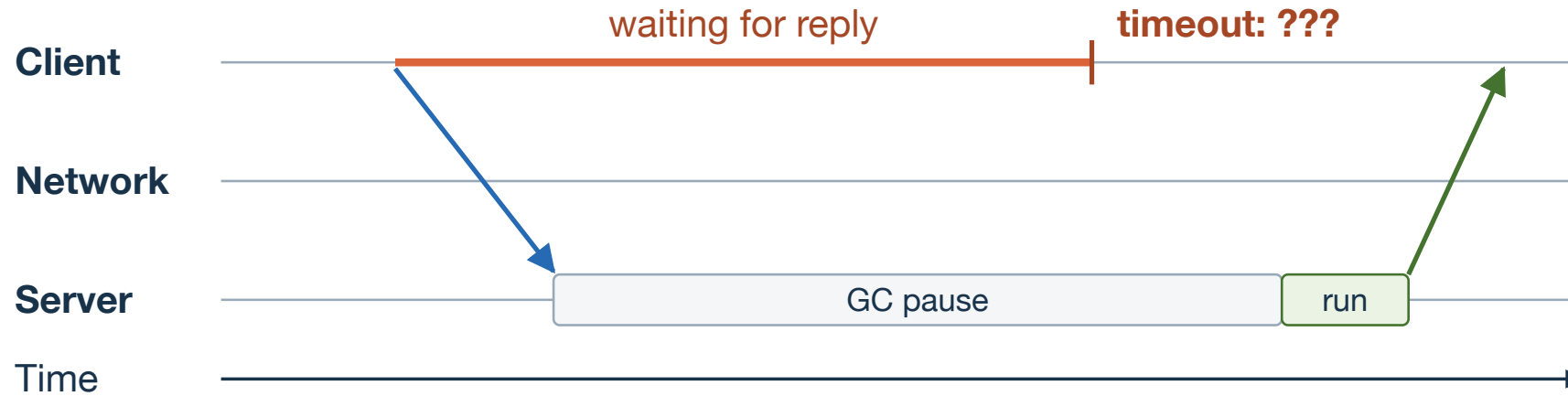
3. The remote node may have **failed** (e.g., it crashed or it was powered down)

¹ Adapted from Kleppmann, Designing Data-Intensive Applications, Figure 8-1

When RPC meets failures

4. Server paused

1



A user sends an RPC but the server **does not reply**; possible reasons:

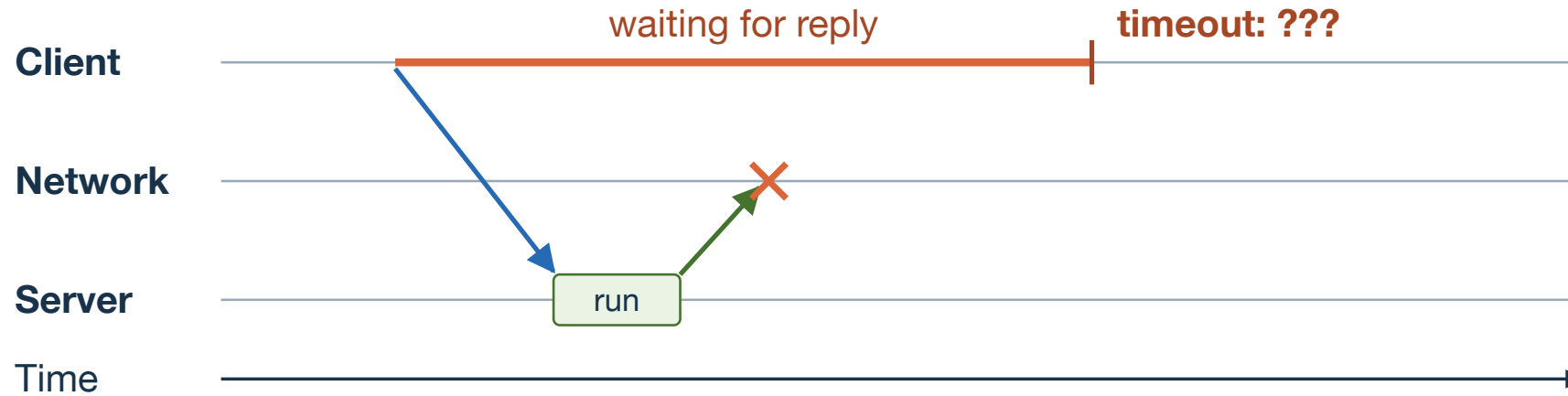
4. The remote node may have **temporarily stopped** responding (e.g., it is experiencing a long **garbage collection pause**)

¹ Adapted from Kleppmann, Designing Data-Intensive Applications, Figure 8-1

When RPC meets failures

5. Reply lost

1



A user sends an RPC but the server **does not reply**; possible reasons:

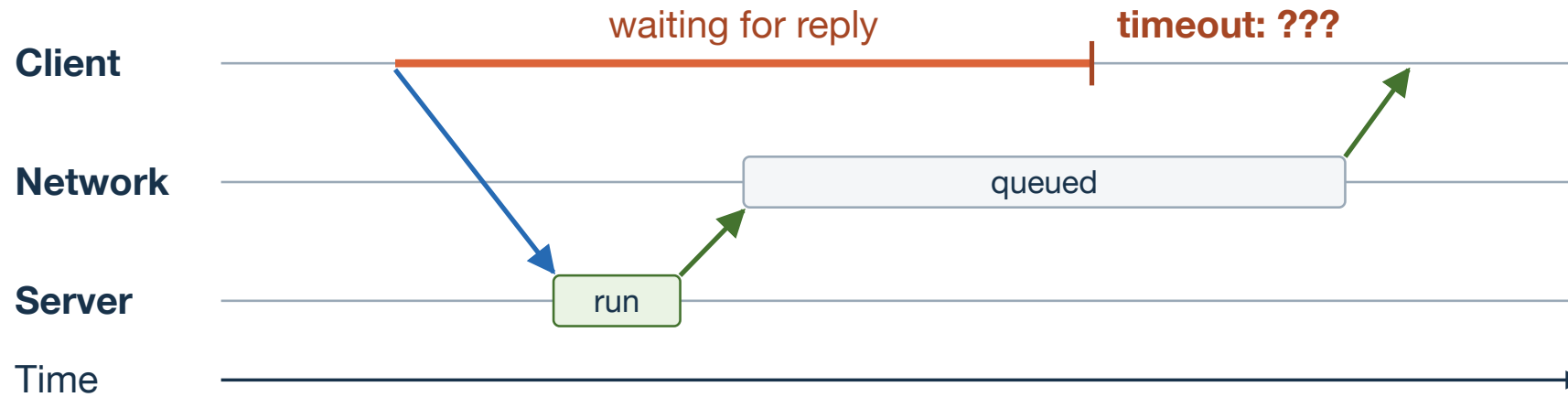
5. The remote node may have **executed the function**, but the response has been **lost** on the network (e.g., a network switch has been misconfigured)

¹ Adapted from Kleppmann, Designing Data-Intensive Applications, Figure 8-1

When RPC meets failures

6. Reply delayed

1



A user sends an RPC but the server **does not reply**; possible reasons:

6. The remote node may have **executed the function**, but the response has been **delayed** and will be delivered later (e.g., the network or your own machine is overloaded)

RPC != PC

The **transparency** of RPC breaks here

- Applications should be prepared to deal with RPC failure
- E.g., gRPC returns a `Status` for the application to check

```
// Here we can use the stub's newly available method we just added.
Status status = stub_->SayHelloAgain(&context, request, &reply);
if (status.ok()) {
    return reply.message();
} else {
    std::cout << status.error_code() << ": " << status.error_message()
              << std::endl;
    return "RPC failed";
}
```

When RPC meets failures

Semantics of remote procedure calls

- Local procedure call: exactly once

A remote procedure call may be executed

- 0 times: the server crashed or the server process died before executing the server code
- 1 time: everything worked well, as expected
- 1 or more times: excess latency or lost reply from the server, and client retransmission
- 0 or 1 time: the function executes at most once

What is the most desirable RPC semantics for the application developers?

What is the desirable RPC semantics for the system developers?

RPC semantics

Most RPC systems offer either

- At-least-once semantics
- At-most-once semantics

Simple retransmission leads to "at-least-once"

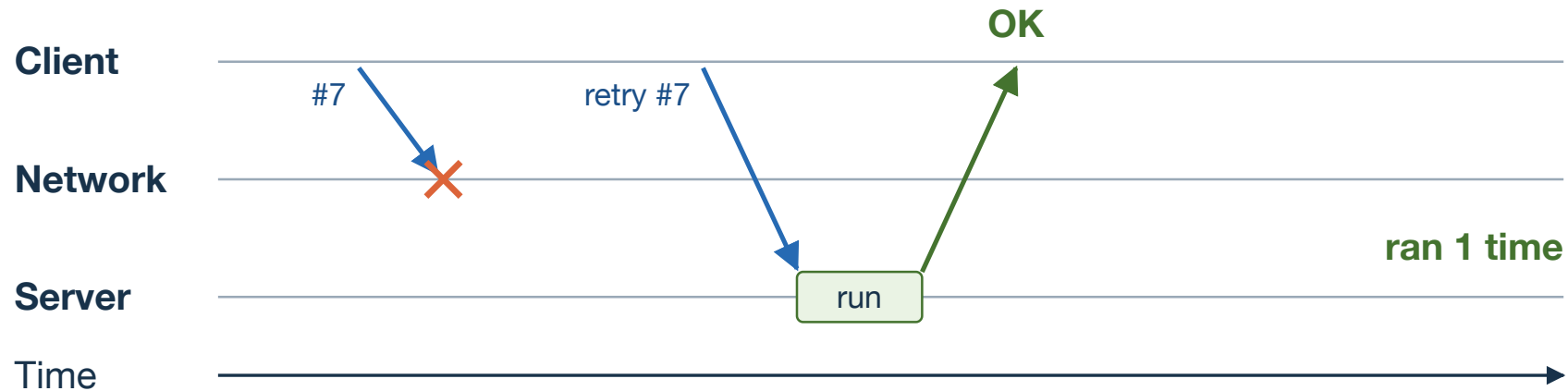
Birrell's RPC semantics: how

Birrell's RPC semantics (1984): **at-most-once**

- Server says **OK**: executed exactly once
- Server says **CRASH**: executed zero times or once; the caller is not told which

How: each call carries a **call ID**; the server remembers the **last call ID** of each client

1. Request lost: client retransmits



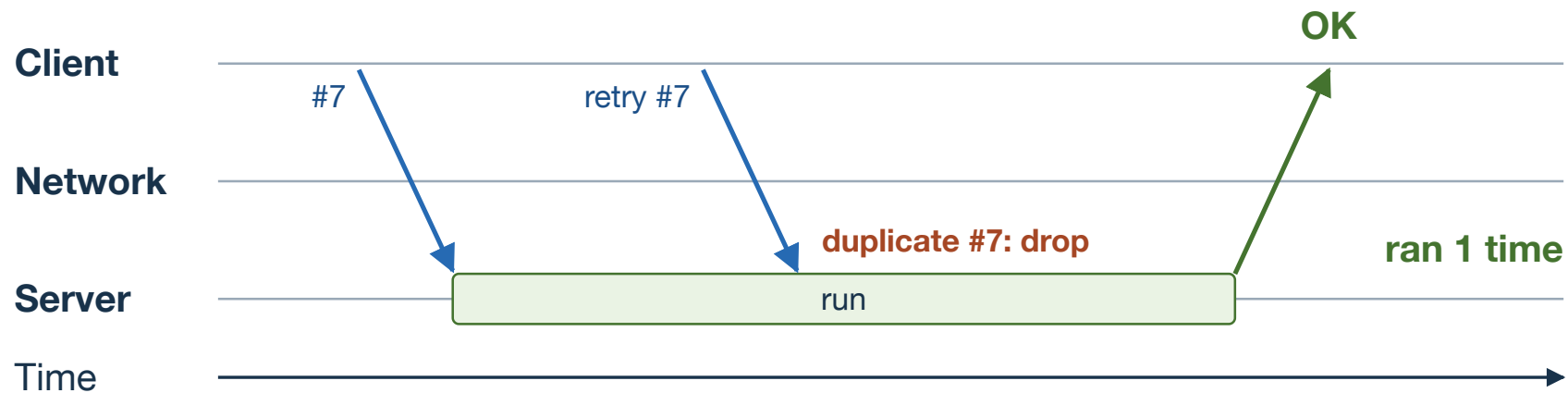
Birrell's RPC semantics: how

Birrell's RPC semantics (1984): **at-most-once**

- Server says **OK**: executed exactly once
- Server says **CRASH**: executed zero times or once; the caller is not told which

How: each call carries a **call ID**; the server remembers the **last call ID** of each client

2. Duplicate request: server drops it



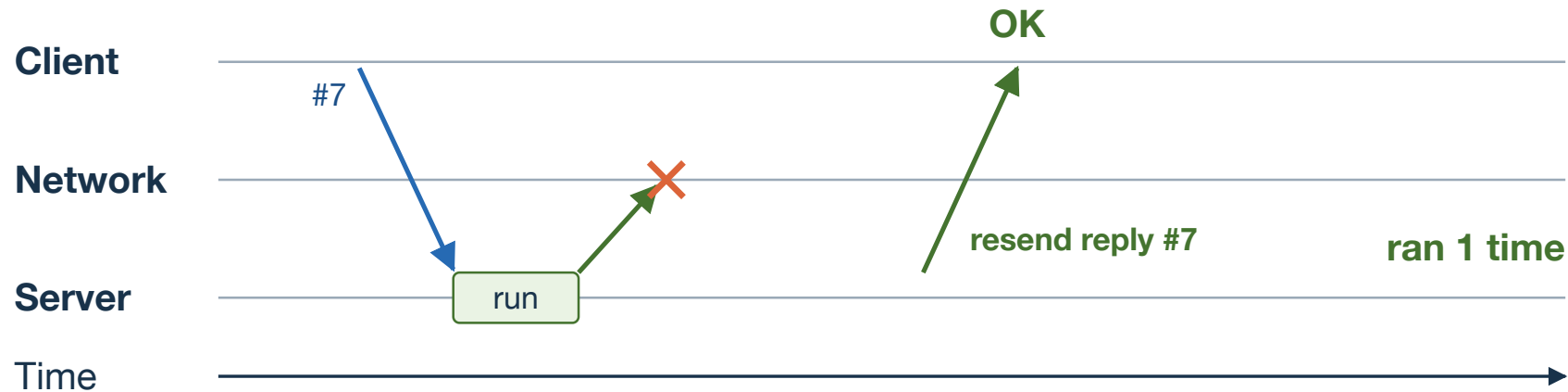
Birrell's RPC semantics: how

Birrell's RPC semantics (1984): **at-most-once**

- Server says **OK**: executed exactly once
- Server says **CRASH**: executed zero times or once; the caller is not told which

How: each call carries a **call ID**; the server remembers the **last call ID** of each client

3. Reply lost: server resends the reply



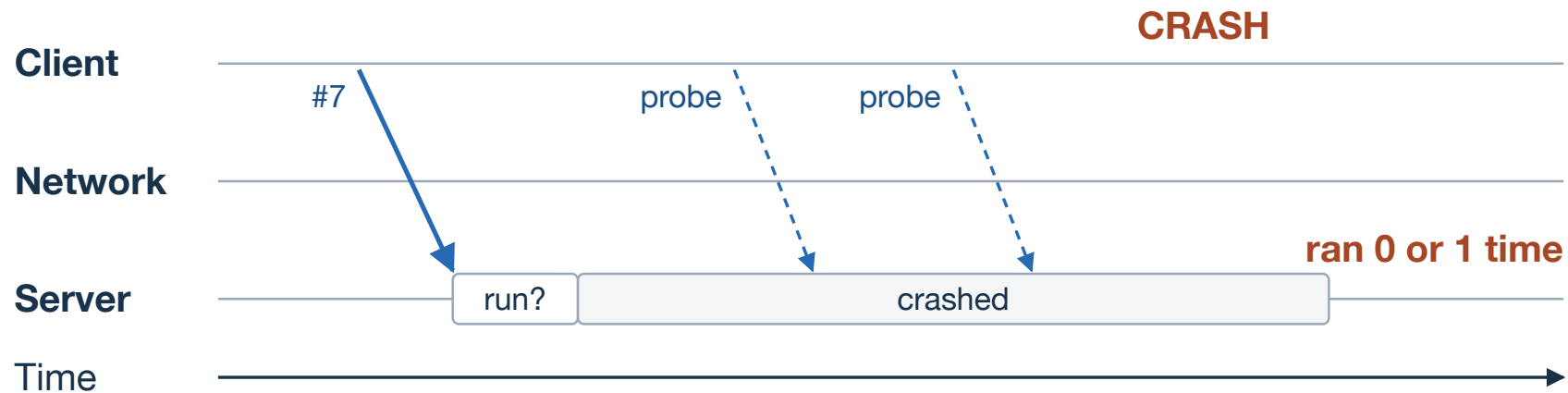
Birrell's RPC semantics: how

Birrell's RPC semantics (1984): **at-most-once**

- Server says **OK**: executed exactly once
- Server says **CRASH**: executed zero times or once; the caller is not told which

How: each call carries a **call ID**; the server remembers the **last call ID** of each client

4. Server crashed: client gets CRASH

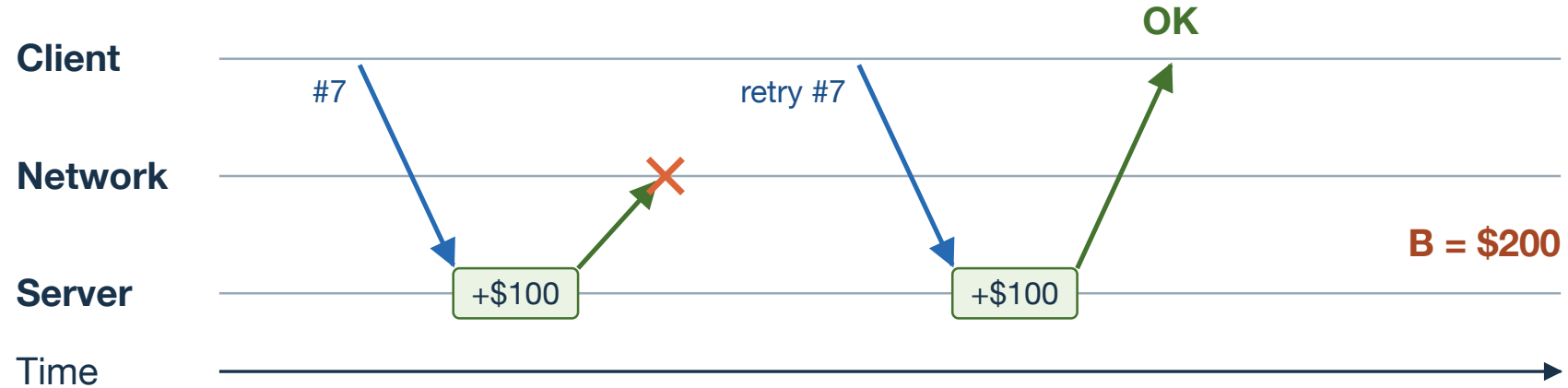


Duplicates are dropped, lost replies are resent: never run twice

Birrell's RPC semantics: why useful

Example: `Transfer($100)` to account B, which starts at \$0

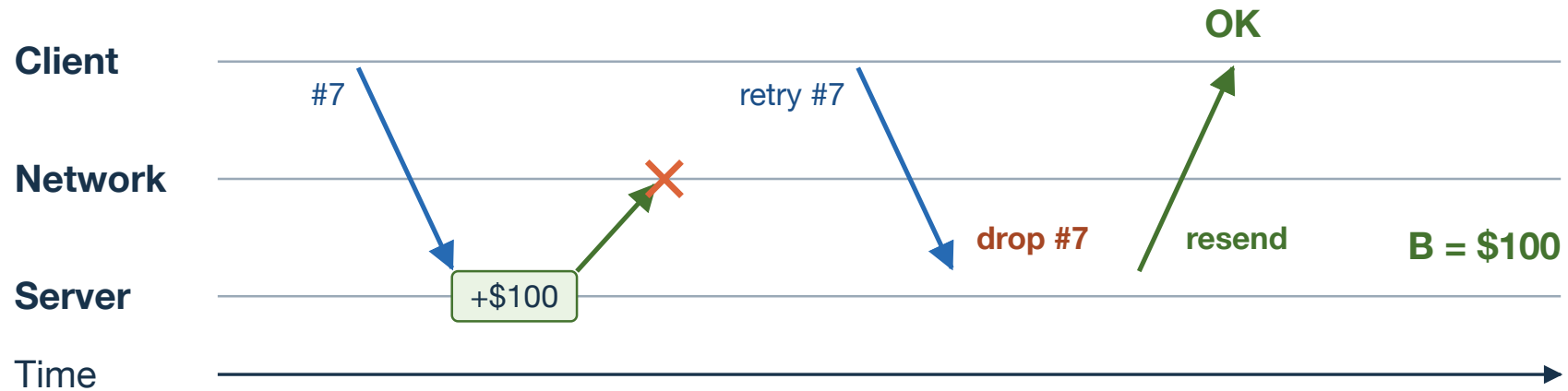
At-least-once: reply lost, client retries



Birrell's RPC semantics: why useful

Example: `Transfer($100)` to account B, which starts at \$0

Birrell: reply lost, client retries

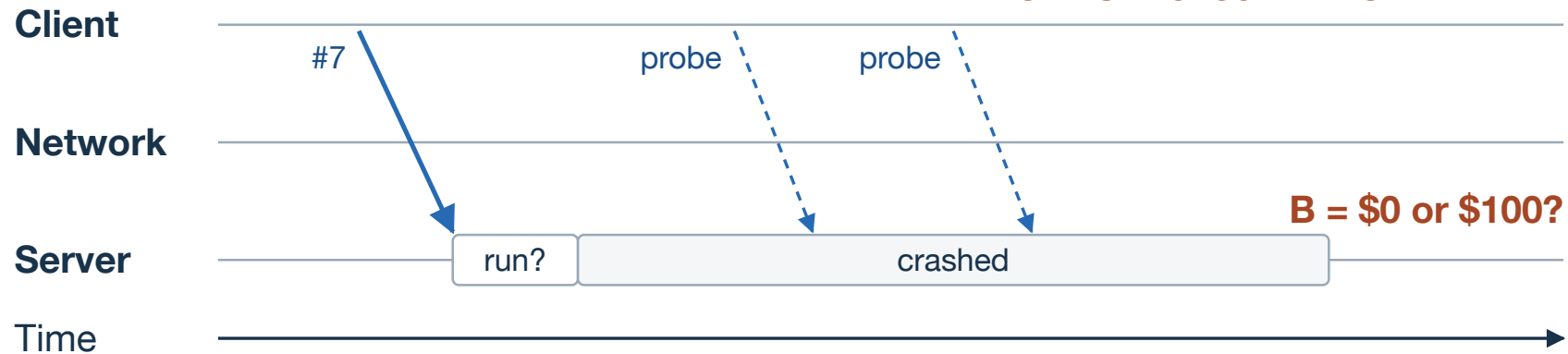


Birrell's RPC semantics: why useful

Example: `Transfer($100)` to account B, which starts at \$0

Birrell: server crashed

CRASH: check B first



- At-least-once: OK may still mean **run twice**, safe only for idempotent calls
- Birrell: **OK means run once**. Only CRASH needs app-level checking

Much easier than exactly-once, more useful than at-least-once

RPC semantics

Understand the application

- **Idempotent**: may be run any number of times without harm (e.g., `i = 1`)
- **Non-idempotent**: those with side effects (e.g., `i++`)

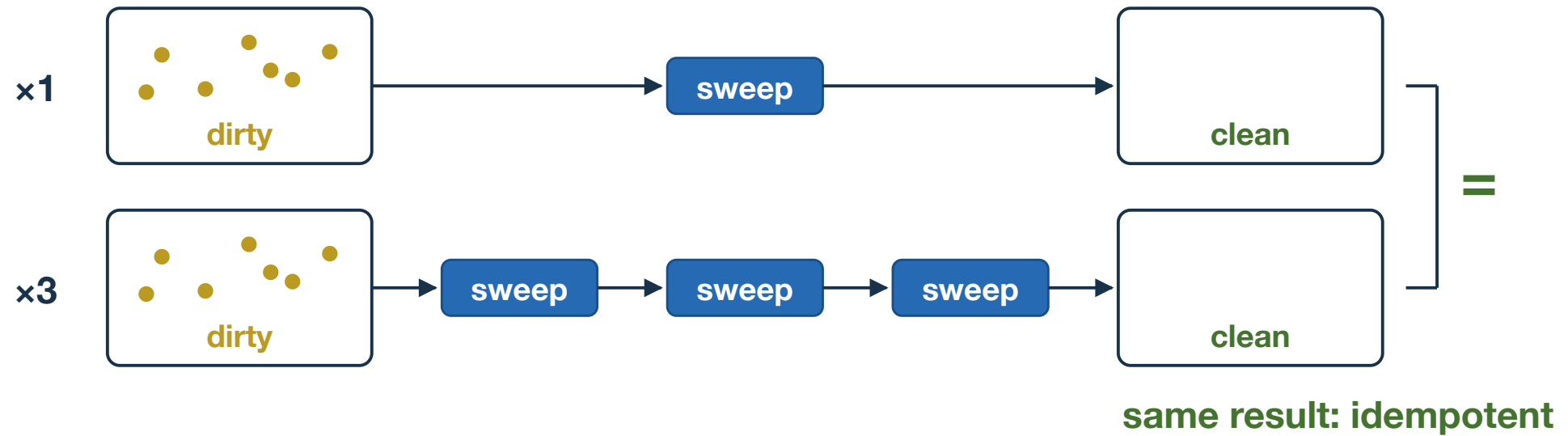
When is at-least-once **OK**?

- If there are no side effects (e.g., read-only operations)
- If the app has its own plan for detecting duplication

Cases of idempotence

“Idempotence means that multiple invocations of some work are identical to exactly one invocation.”
— Pat Helland

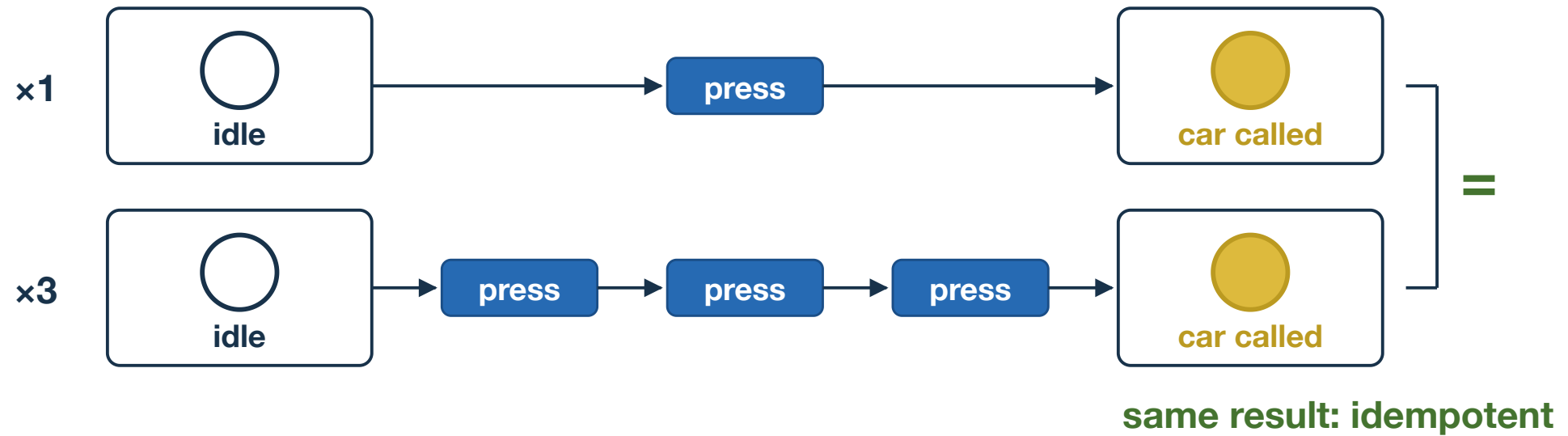
Sweep the floor



Cases of idempotence

“Idempotence means that multiple invocations of some work are identical to exactly one invocation.”
— Pat Helland

Press the elevator button

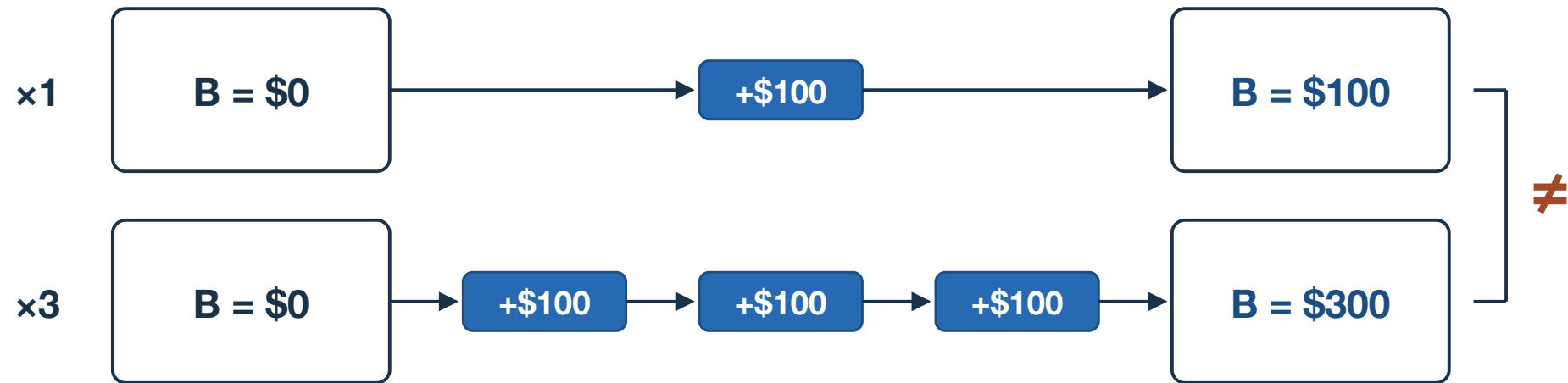


Cases of idempotence

"Idempotence means that multiple invocations of some work are identical to exactly one invocation."

— Pat Helland

Deposit \$100 (+)



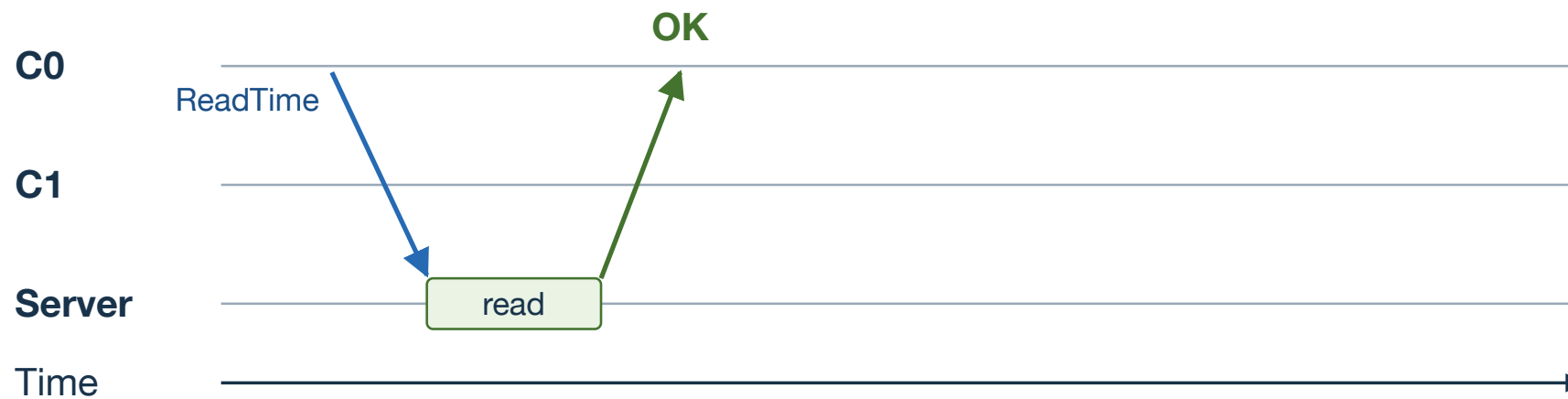
\$100 vs. \$300: non-idempotent

Idempotence in RPC

```
RPC client  
Call(server, "ReadTime");
```

```
RPC server  
ReadTime() {  
    return current_time;  
}
```

C0 calls ReadTime

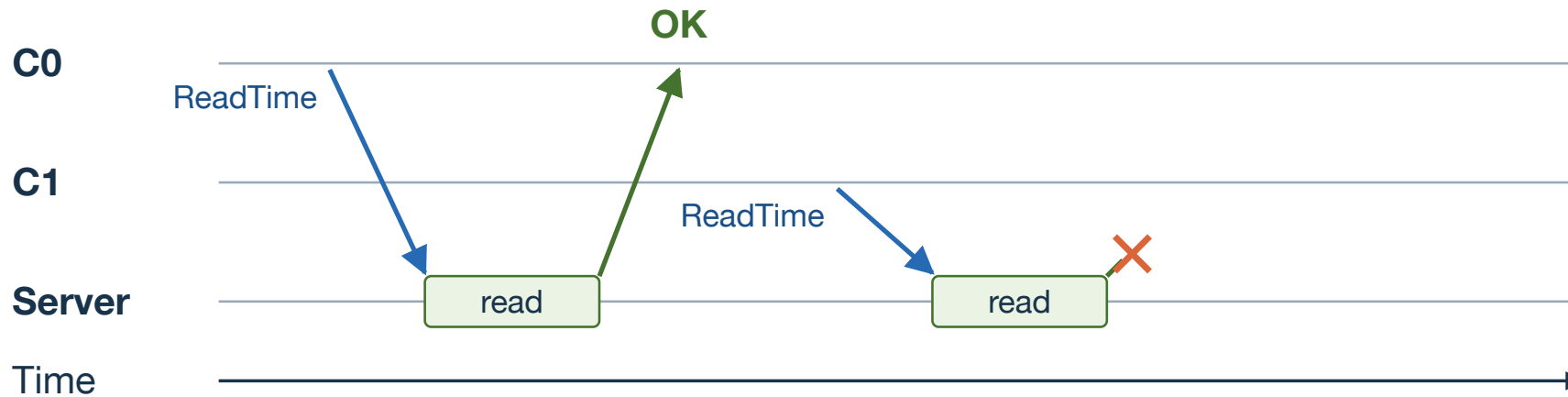


Idempotence in RPC

```
RPC client  
Call(server, "ReadTime");
```

```
RPC server  
ReadTime() {  
    return current_time;  
}
```

C1 calls ReadTime: reply lost



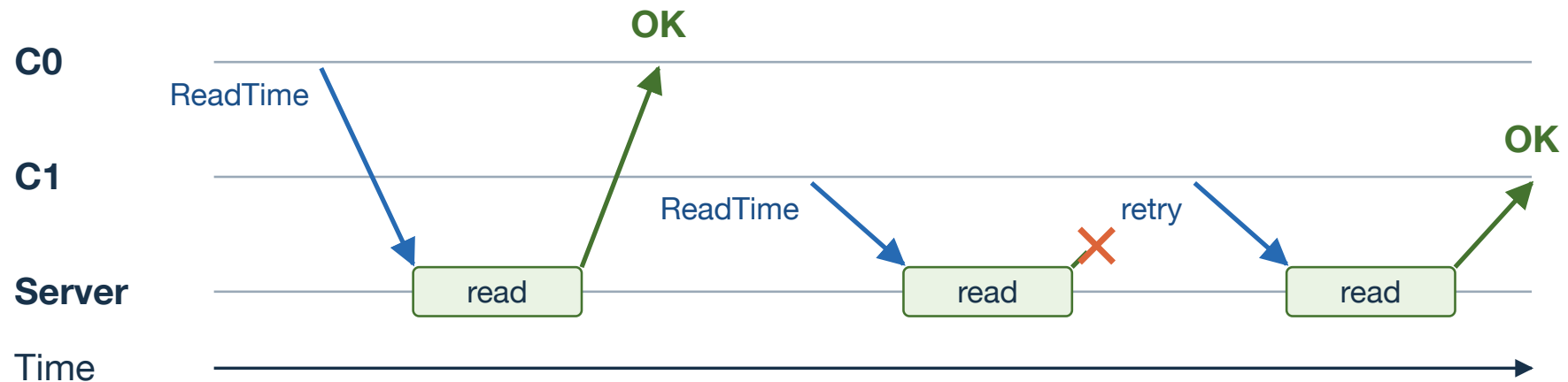
Idempotence in RPC

```
RPC client  
Call(server, "ReadTime");
```

```
RPC server  
ReadTime() {  
    return current_time;  
}
```

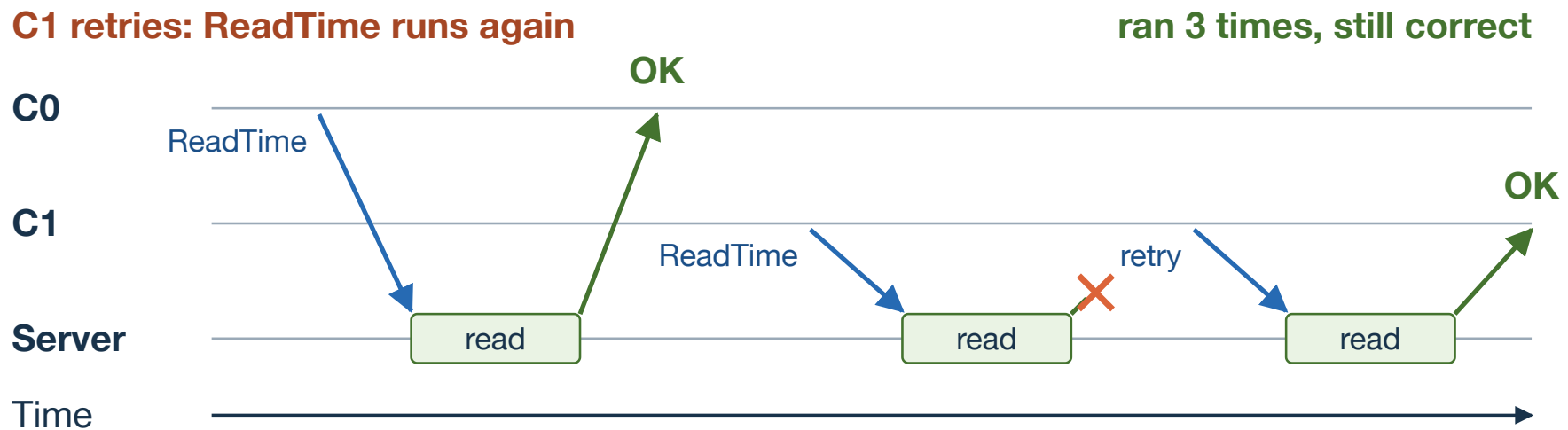
C1 retries: ReadTime runs again

ran 3 times, still correct



Idempotence in RPC

Expect ReadTime to be called twice, but it is actually called 3 times
Is it OK? Why?

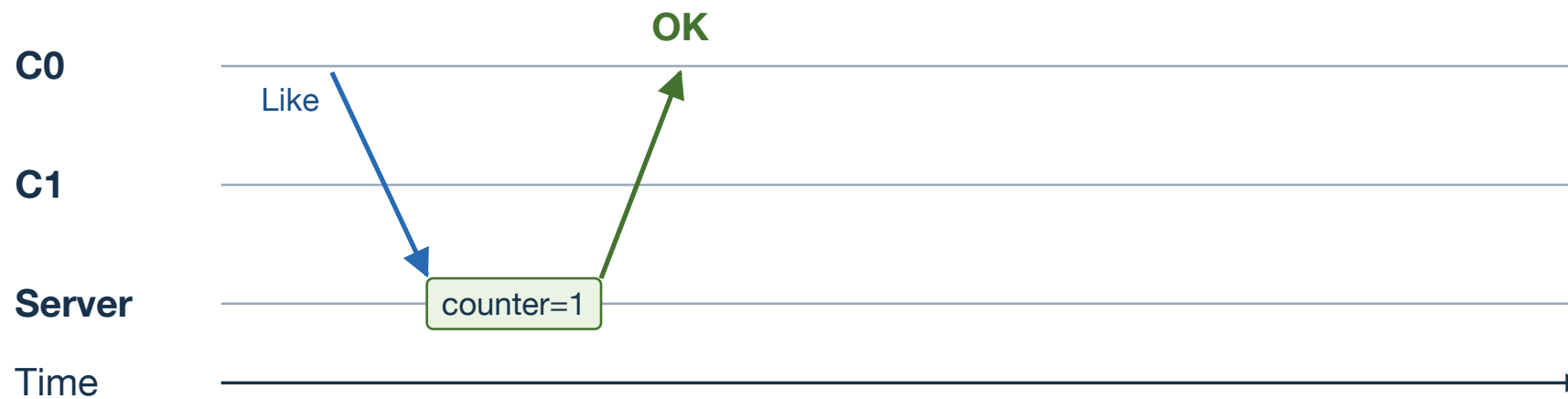


Idempotence in RPC

```
RPC client  
Call(server, "Like");
```

```
RPC server  
Like() {  
    counter++;  
}
```

C0 calls Like

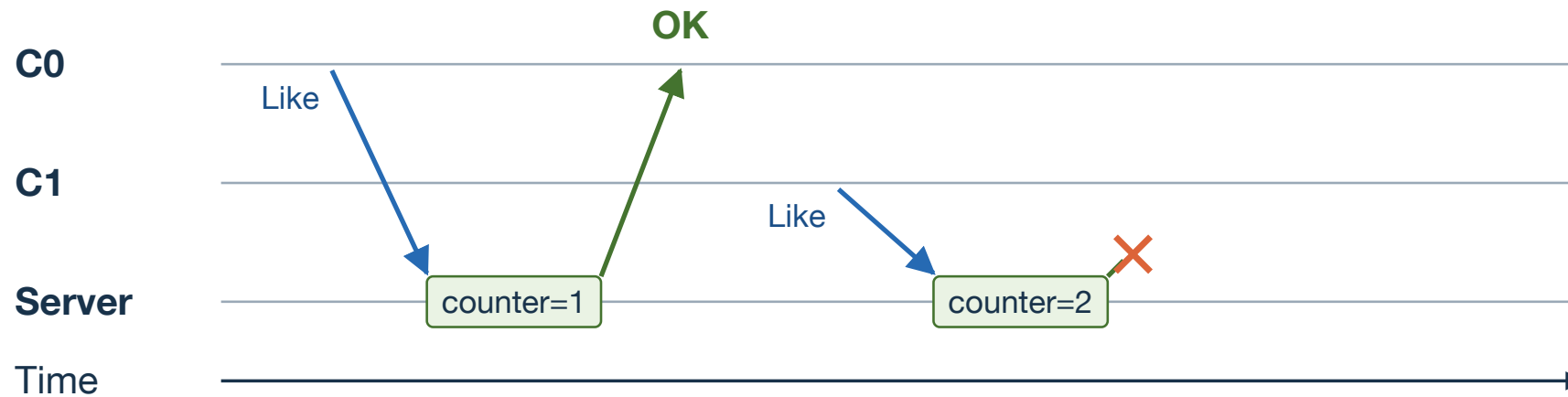


Idempotence in RPC

```
RPC client  
Call(server, "Like");
```

```
RPC server  
Like() {  
    counter++;  
}
```

C1 calls Like: reply lost



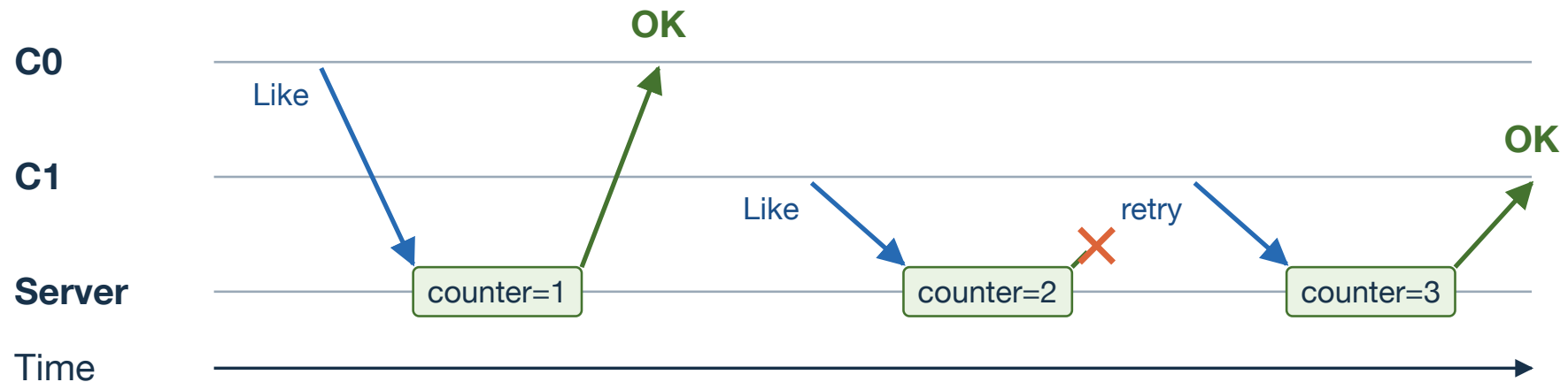
Idempotence in RPC

```
RPC client  
Call(server, "Like");
```

```
RPC server  
Like() {  
    counter++;  
}
```

C1 retries: Like runs again

counter = 3, expected 2



Idempotence in RPC

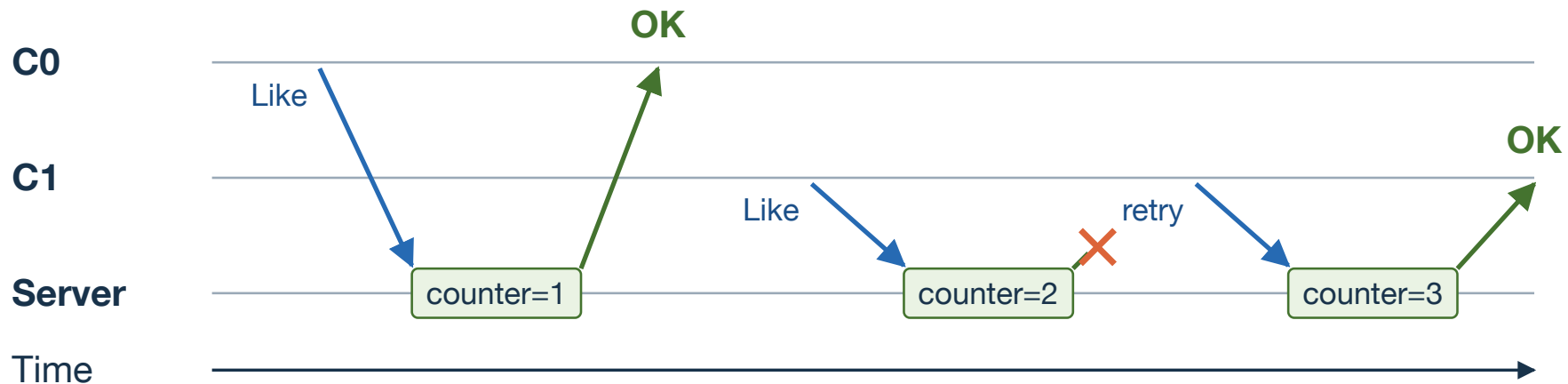
Expect Like to be called **twice**, but it is actually called **3 times**

- Is it OK? Why?
- What about something like **Vote**?



C1 retries: Like runs again

counter = 3, expected 2



RPC semantics

Understand the application

- **Idempotent**: may be run any number of times without harm
- **Non-idempotent**: those with side effects (e.g., call `i++` at the server)

When is at-least-once **OK**?

- If there are no side effects (e.g., read-only operations, `ReadTime`)
- If the app has its own plan for detecting duplication
 - E.g., record how many times a counter has been incremented

Ideal RPC semantics: exactly-once

Like a single-machine function call

Implement exactly-once semantics

- The server remembers the requests it has seen and the replies to executed RPCs (needs to survive reboots)
- Detect duplicates: requests need unique IDs (xids)

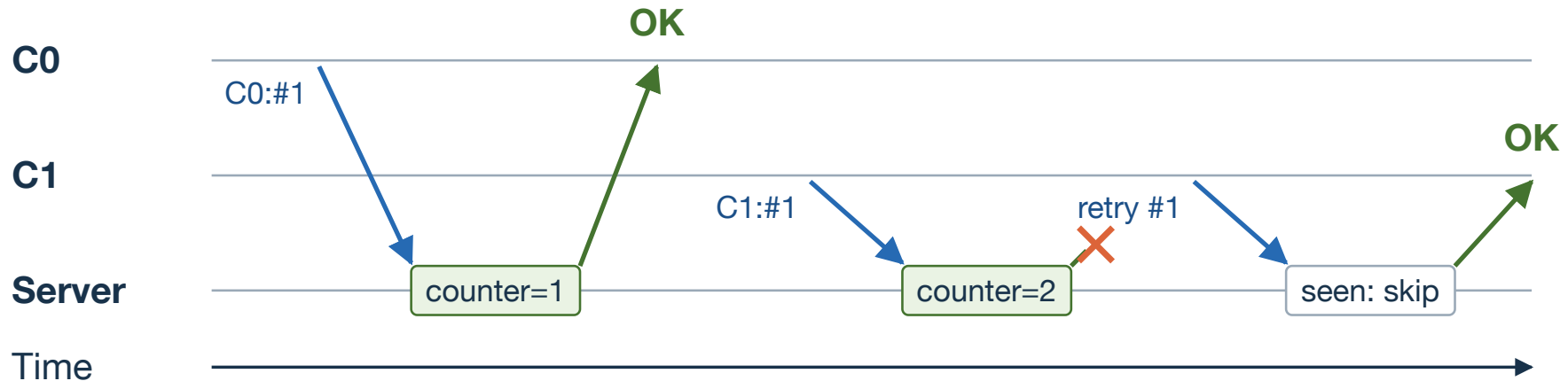
Idempotence in RPC

```
RPC client  
Call(server, "Like");
```

```
RPC server  
Like(xid) {  
    if xid not seen {  
        counter++;  
    }  
}
```

C1 retries with the same xid: server skips it

counter = 2



Ideal RPC semantics: exactly-once

Like a single-machine function call

Implement exactly-once semantics

- The server remembers the requests it has seen and the replies to executed RPCs (across reboots)
- Detect duplicates: requests need unique IDs (xids)

Assumption: failures are eventually repaired, and the client retries forever

- How to correctly recover from failure? See later lectures

Put it all together: RPC system components

1. Standards for the wire format of RPC messages and data types
2. A library of routines to marshal / unmarshal data
3. A stub generator, or RPC compiler, to produce "stubs"
 - For the client: marshal arguments, call, wait, unmarshal the reply
 - For the server: unmarshal arguments, call the real function, marshal the reply

Put it all together: RPC system components

4. Server framework

- Dispatch each call message to the correct server stub
- Remember executed calls if it provides **at-most-once** or **exactly-once** semantics

5. Client framework

- Give each reply to the correct waiting thread / callback
- Retry on timeout or server crash

6. **Binding**: how does the client find the right server? We typically need a service

Case study: Sun RPC / reply cache

A reply cache provides **at-most-once** protection while the saved reply remains cached.

```
// Simplified svc_dg_recv: cache-hit path  
if reply_cache contains xid:  
    send reply_cache[xid]  
    return // do not call the handler  
call the handler
```

Back to **Like()**: a cache hit means **no second counter++**.
After eviction or restart, a retry **may execute again**.¹

- Open in VS Code: [svc_dg.c:224](#)

¹ Simplified from libtirpc/src/svc_dg.c:224–234. Cache must be enabled; the real key includes caller and method identity. ONC RPC semantics: RFC 5531, Section 5 (<https://www.rfc-editor.org/rfc/rfc5531#section-5>).

Case study: gRPC / Protobuf

Protobuf uses **field tags** and **varints** to reduce the bytes needed for small integer fields.

```
message Person {  
  optional int64 favorite_number = 2;  
}  
// favorite_number = 1337  
// Encoded field: 10 | B9 0A (hex)  
//           tag  value
```

Back to **favoriteNumber = 1337**: the integer uses **2 bytes instead of 8**, plus a 1-byte field tag.¹

- [Open in VS Code: wire.go:185](https://github.com/grpc/grpc-go/blob/master/encoding/proto/proto.go#L185)

¹ Same Person example as the encoding section. Schema/bytes are a teaching example, not a full RPC packet. [grpc-go/encoding/proto/proto.go:43](https://github.com/grpc/grpc-go/blob/master/encoding/proto/proto.go#L43); [protobuf-go/encoding/protowire/wire.go:185-195](https://github.com/grpc/grpc-go/blob/master/encoding/protowire/wire.go#L185-195).

Case study: gRPC / retry

A retry policy gives a call **another chance after a transient failure**.

```
// Example retryPolicy for the ReadTime method
"maxAttempts": 3,
"initialBackoff": "0.1s",
"maxBackoff": "1s",
"backoffMultiplier": 2,
"retryableStatusCodes": ["UNAVAILABLE"]
```

Back to **ReadTime() vs. Like()**: retrying the read is safe; `Like()` can increment twice.¹

- [Open in VS Code: main.go:41](#)

¹ Service-config fragment; values are illustrative. maxAttempts includes the first attempt. Based on `grpc-go/examples/features/retry/client/main.go`; retry remains subject to deadline and commit rules.

References

- [1] [Implementing Remote Procedure Calls](#) — Birrell & Nelson, 1984. ACM TOCS 2(1) (P2, P41, P42)
- [2] [Principles of Computer System Design: An Introduction](#) — Saltzer & Kaashoek, 2009. Morgan Kaufmann (P6, P9, P12, P14, P15)
- [3] [Designing Data-Intensive Applications](#) — Martin Kleppmann, 2017. O'Reilly Media (P22, P24, P26, P27, P28, P29, P30, P31, P37)
- [4] [gRPC C++ Quick start](#) — gRPC Authors (P38)
- [5] [Idempotence Is Not a Medical Condition](#) — Pat Helland, 2012. ACM Queue 10(4) (P44)